

Spring Boot

Violet

2026-04-15

Chapter 0

目 录

I	Spring Boot 基础	
1.	初识 Spring Boot	9
1.1.	Spring Boot 诞生背景与核心价值	9
1.1.1.	传统 Spring 开发的痛点	9
1.1.2.	Spring Boot 的解决方案	9
1.1.3.	Spring Boot 的核心价值	10
1.2.	Spring Boot 版本演进与选型建议	10
1.2.1.	版本历史	10
1.2.2.	Spring Boot 3.x 重大变化	10
1.2.3.	版本选型建议	11
1.3.	开发环境准备	11
1.3.1.	JDK 安装与配置	11
1.3.2.	Maven 安装	12
1.3.3.	Gradle 安装（可选）	13
1.3.4.	IDE 选择	13
1.4.	使用 Spring Initializr 创建第一个项目	13
1.4.1.	什么是 Spring Initializr	13
1.4.2.	在线方式创建项目	14
1.4.3.	命令行方式	14
1.4.4.	IntelliJ IDEA 集成	14
1.5.	项目结构解析与核心注解	15
1.5.1.	标准项目结构	15
1.5.2.	@SpringBootApplication 注解	15
1.5.3.	常用注解速查	16
1.6.	内嵌 Web 容器与启动原理初探	16
1.6.1.	内嵌 Web 容器	16
1.6.2.	启动流程简析	17
1.6.3.	第一个 REST 接口	18
2.	日志系统详解	19
2.1.	Java 日志生态与 SLF4J 门面模式	19
2.1.1.	Java 日志生态演变	19
2.1.2.	SLF4J 的解决方案	19

2.2. Spring Boot 默认日志框架	20
2.2.1. 默认日志框架	20
2.2.2. 日志桥接 (Bridge)	21
2.3. Logback 配置详解	21
2.3.1. 配置文件位置	21
2.3.2. 基本配置结构	22
2.3.3. 日志级别	22
2.3.4. 日志格式模式	23
2.3.5. application.properties 配置	24
2.4. 异步日志与性能优化	24
2.4.1. 异步日志	24
2.4.2. 条件配置 (Profile)	25
2.5. 结构化日志与 JSON 格式输出	25
2.5.1. 为什么需要结构化日志	25
2.5.2. JSON 格式化日志	26
2.6. MDC 与链路追踪上下文	27
2.6.1. MDC (Mapped Diagnostic Context)	27
2.7. 敏感信息脱敏与安全日志	29
2.7.1. 为什么需要脱敏	29
2.7.2. 实现方式	29
2.8. 多环境日志配置	29
2.8.1. 基于 Profile 的配置	29
2.8.2. 动态调整日志级别	30
2.9. 自定义 Appender 与日志扩展	30
2.9.1. 自定义 Appender	30
2.9.2. 最佳实践总结	31

II

Spring Boot 核心

3. 定时任务与异步处理	34
3.1. 定时任务基础	34
3.1.1. @Scheduled 注解详解	34
3.1.2. Cron 表达式语法	35
3.1.3. 固定速率与固定延迟	36
3.2. 异步处理核心	37
3.2.1. @Async 注解使用	37
3.2.2. 线程池配置与管理	38
3.2.3. 异步返回值 Future	41
3.3. 高级定时任务	44
3.3.1. 动态定时任务	44
3.3.2. 分布式定时任务	46

3.3.3. Quartz 框架集成 ·····	53
3.4. 异步编程最佳实践 ·····	58
3.4.1. 异常处理机制 ·····	58
3.4.2. 上下文传递 ·····	60
3.4.3. 性能调优与监控 ·····	63
3.5. 本章小结 ·····	66
3.5.1. 定时任务 ·····	66
3.5.2. 异步处理 ·····	66
3.5.3. 最佳实践 ·····	66
4. 日志系统详解 ·····	68
4.1. Java 日志生态与 SLF4J 门面模式 ·····	68
4.1.1. Java 日志生态演变 ·····	68
4.1.2. SLF4J 的解决方案 ·····	68
4.2. Spring Boot 默认日志框架 ·····	69
4.2.1. 默认日志框架 ·····	69
4.2.2. 日志桥接 (Bridge) ·····	70
4.3. Logback 配置详解 ·····	70
4.3.1. 配置文件位置 ·····	70
4.3.2. 基本配置结构 ·····	71
4.3.3. 日志级别 ·····	71
4.3.4. 日志格式模式 ·····	72
4.3.5. application.properties 配置 ·····	73
4.4. 异步日志与性能优化 ·····	73
4.4.1. 异步日志 ·····	73
4.4.2. 条件配置 (Profile) ·····	74
4.5. 结构化日志与 JSON 格式输出 ·····	74
4.5.1. 为什么需要结构化日志 ·····	74
4.5.2. JSON 格式化日志 ·····	75
4.6. MDC 与链路追踪上下文 ·····	76
4.6.1. MDC (Mapped Diagnostic Context) ·····	76
4.7. 敏感信息脱敏与安全日志 ·····	78
4.7.1. 为什么需要脱敏 ·····	78
4.7.2. 实现方式 ·····	78
4.8. 多环境日志配置 ·····	78
4.8.1. 基于 Profile 的配置 ·····	78
4.8.2. 动态调整日志级别 ·····	79
4.9. 自定义 Appender 与日志扩展 ·····	79
4.9.1. 自定义 Appender ·····	79
4.9.2. 最佳实践总结 ·····	80

5. 监控与可观测性	82
5.1. 可观测性三大支柱	82
5.1.1. Logs (日志)	82
5.1.2. Metrics (指标)	82
5.1.3. Traces (链路追踪)	83
5.1.4. 三大支柱的关系	83
5.2. Actuator 端点详解	83
5.2.1. 什么是 Actuator	83
5.2.2. 启用 Actuator	84
5.2.3. 常用端点	84
5.2.4. health 端点	84
5.2.5. info 端点	85
5.2.6. metrics 端点	86
5.3. 自定义健康检查	86
5.3.1. HealthIndicator 接口	86
5.3.2. AbstractHealthIndicator	87
5.3.3. 复合健康检查	88
5.4. 自定义指标收集	89
5.4.1. MeterRegistry	89
5.4.2. Counter (计数器)	89
5.4.3. Gauge (仪表盘)	90
5.4.4. Timer (计时器)	91
5.4.5. DistributionSummary (分布摘要)	92
5.5. Micrometer 核心概念与抽象层	92
5.5.1. Micrometer 是什么	92
5.5.2. 架构设计	93
5.5.3. 核心概念	93
5.6. 集成 Prometheus 与 Grafana 可视化	94
5.6.1. Prometheus 简介	94
5.6.2. Spring Boot 集成 Prometheus	94
5.6.3. Prometheus 部署	95
5.6.4. PromQL 查询语言	96
5.6.5. Grafana 可视化	97
5.7. 链路追踪基础	98
5.7.1. 为什么需要链路追踪	98
5.7.2. OpenTelemetry	98
5.7.3. Spring Boot 集成 Micrometer Tracing	98
5.7.4. 手动创建 Span	99
5.7.5. Trace ID 传播	100
5.8. 分布式追踪实践	100
5.8.1. Zipkin 部署	100

5.8.2. Jaeger 部署 101

5.8.3. 追踪数据分析 102

5.9. 日志聚合与分析 102

5.9.1. 为什么需要日志聚合 102

5.9.2. ELK Stack 103

5.9.3. Loki + Grafana 104

5.10. 告警与通知 105

5.10.1. Prometheus Alertmanager 105

5.10.2. Grafana 告警 106

5.10.3. 钉钉告警配置 107

III

微服务与中间件

6. 微服务架构基础 109

6.1. 微服务架构概述 109

6.2. 服务注册与发现 109

6.3. 配置中心 109

6.4. API 网关 109

6.5. 负载均衡与熔断降级 109



Spring Boot 基础

1. 初识 Spring Boot	9
2. 日志系统详解	19

Chapter 1

初识 Spring Boot

NOTE

版本说明：本笔记基于以下技术栈版本编写，示例代码均在此环境下测试通过。

- Spring Boot: 3.2.4
- JDK: 17 (LTS)
- Kotlin: 2.1.0
- Maven: 3.9+
- Gradle: 8.5+
- MySQL: 8.0+
- Redis: 7.0+

1 Spring Boot 诞生背景与核心价值

1.1 传统 Spring 开发的痛点

在 Spring Boot 出现之前，使用 Spring 框架开发应用存在诸多问题：

配置复杂：

- 大量的 XML 配置文件
- 依赖管理繁琐
- 需要手动配置 Tomcat 等 Web 容器

部署困难：

- 需要打包成 WAR 文件
- 部署到外部应用服务器
- 环境配置复杂

开发效率低：

- 项目搭建耗时长
- 约定不明确
- 学习曲线陡峭

1.2 Spring Boot 的解决方案

Spring Boot 的核心理念是“约定优于配置”（Convention over Configuration）。

核心特性：

特性	说明	优势
自动配置	根据依赖自动配置 Bean	零配置启动

内嵌容器	内置 Tomcat/Jetty/Undertow	独立运行
Starter 依赖	一站式依赖管理	简化 Maven/Gradle
生产就绪	Actuator 监控端点	开箱即用
无代码生成	无需 XML, 无需代码生成	简洁清晰

1.3 Spring Boot 的核心价值

1. 快速启动:

- 几分钟内创建可运行的应用
- 减少样板代码
- 专注业务逻辑

2. 简化配置:

- 自动配置大多数场景
- 配置文件简单明了
- 支持多种配置格式

3. 独立运行:

- 内嵌 Web 容器
- JAR 包直接运行
- 微服务友好

4. 生态丰富:

- 大量 Starter 可用
- 社区活跃
- 文档完善

TIP

Spring Boot 不是要替代 Spring, 而是让 Spring 更容易使用。

2 Spring Boot 版本演进与选型建议

2.1 版本历史

版本	发布年份	JDK 要求	状态
1.x	2014	Java 6+	已停止维护
2.x	2018	Java 8+	部分维护
3.x	2022	Java 17+	当前主流

2.2 Spring Boot 3.x 重大变化

Spring Boot 3.x 是基于 Spring Framework 6 的重大版本升级:

核心变化:

1. 最低 JDK 版本：从 Java 8 提升到 Java 17
2. Jakarta EE：从 `javax.*` 迁移到 `jakarta.*`
3. GraalVM 原生镜像：原生支持 AOT 编译
4. Observability：内置可观测性支持
5. Kotlin 协程：更好的响应式支持

Jakarta EE 迁移

```
1 // Spring Boot 2.x (javax)
2 import javax.servlet.http.HttpServletRequest;
3 import javax.persistence.Entity;
4
5 // Spring Boot 3.x (jakarta)
6 import jakarta.servlet.http.HttpServletRequest;
7 import jakarta.persistence.Entity;
```

CAUTION

从 2.x 升级到 3.x 时，所有 `javax.*` 包都需要改为 `jakarta.*`，这是破坏性变更。

2.3 版本选型建议

新项目：

- 推荐：Spring Boot 3.2+ + JDK 17/21
- 理由：长期支持、性能更好、新特性

存量项目：

- Spring Boot 2.7.x：可以继续使用，但建议规划升级
- Spring Boot 2.x 以下：强烈建议升级

NOTE

本笔记所有示例基于 Spring Boot 3.2.4 和 JDK 17 编写。

3 开发环境准备

3.1 JDK 安装与配置

推荐版本

- JDK 17 (LTS)：Spring Boot 3.x 最低要求
- JDK 21 (LTS)：最新长期支持版本

安装方式

方式 1：SDKMAN!（推荐，Linux/Mac）

```
1 # 安装SDKMAN!
2 curl -s "https://get.sdkman.io" | bash
```

```
3
4 # 安装JDK 17
5 sdk install java 17.0.10-tem
6
7 # 切换版本
8 sdk use java 17.0.10-tem
```

方式 2：官方下载

访问 <https://adoptium.net/> 下载 Temurin JDK。

方式 3：Homebrew (Mac)

```
1 brew install openjdk@17
```

 Bash

验证安装

```
1 java -version
2 # 输出: openjdk version "17.0.10" ...
3
4 javac -version
5 # 输出: javac 17.0.10
```

 Bash

3.2 Maven 安装

推荐版本

Maven 3.9+

安装方式

SDKMAN!

```
1 sdk install maven 3.9.6
```

 Bash

Homebrew

```
1 brew install maven
```

 Bash

官方下载

访问 <https://maven.apache.org/download.cgi>

配置国内镜像

编辑 `~/.m2/settings.xml`：

```
1 <settings>
2   <mirrors>
3     <mirror>
4       <id>aliyun</id>
5       <mirrorOf>central</mirrorOf>
6       <name>Aliyun Maven</name>
7       <url>https://maven.aliyun.com/repository/public</url>
8     </mirror>
```

 xml

```
9     </mirrors>
10 </settings>
```

验证安装

```
1 mvn -version
2 # 输出: Apache Maven 3.9.6 ...
```

 Bash

3.3 Gradle 安装（可选）

推荐版本

Gradle 8.5+

安装方式

```
1 # SDKMAN!
2 sdk install gradle 8.5
3
4 # Homebrew
5 brew install gradle
```

 Bash

验证安装

```
1 gradle -version
```

 Bash

3.4 IDE 选择

IDE	优点	适用人群
IntelliJ IDEA	Spring 支持最好，智能提示强大	专业开发者
VS Code	轻量级，插件丰富	全栈开发者
Eclipse	免费，老牌 IDE	传统企业

推荐：IntelliJ IDEA Ultimate（付费）或 Community（免费）

IntelliJ IDEA 配置

1. 安装 Spring Boot 插件（Ultimate 版自带）
2. 配置 JDK 17
3. 启用注解处理（Annotation Processing）

4 使用 Spring Initializr 创建第一个项目

4.1 什么是 Spring Initializr

Spring Initializr 是 Spring 官方提供的项目生成工具，可以快速创建 Spring Boot 项目骨架。

访问地址: <https://start.spring.io/>

4.2 在线方式创建项目

步骤

1. 访问 <https://start.spring.io/>
2. 选择项目配置:
 - Project: Maven/Gradle
 - Language: Java/Kotlin
 - Spring Boot: 3.2.4
 - Group: com.example
 - Artifact: demo
 - Package name: com.example.demo
 - Packaging: Jar
 - Java: 17
3. 添加依赖:
 - Spring Web
 - Spring Data JPA
 - MySQL Driver
4. 点击“GENERATE”下载项目
5. 解压并导入 IDE

4.3 命令行方式

使用 curl

```
1 curl https://start.spring.io/starter.zip \
2   -d type=maven-project \
3   -d language=java \
4   -d bootVersion=3.2.4 \
5   -d baseDir=myapp \
6   -d groupId=com.example \
7   -d artifactId=myapp \
8   -d name=myapp \
9   -d packageName=com.example.myapp \
10  -d javaVersion=17 \
11  -d dependencies=web,data-jpa,mysql \
12  -o myapp.zip
13
14 unzip myapp.zip
```

使用 SDKMAN!

```
1 spring init --dependencies=web,data-jpa,mysql myapp
```

4.4 IntelliJ IDEA 集成

1. File → New → Project

2. 选择 Spring Initializr
3. 配置项目信息
4. 选择依赖
5. 完成创建

TIP

推荐使用 IntelliJ IDEA 集成的 Initializr，体验更流畅。

5 项目结构解析与核心注解

5.1 标准项目结构

```
1 myapp/
2 |— src/
3 |   |— main/
4 |   |   |— java/
5 |   |   |   |— com/example/myapp/
6 |   |   |       |— MyAppApplication.java # 启动类
7 |   |   |       |— controller/           # 控制器层
8 |   |   |       |— service/              # 业务逻辑层
9 |   |   |       |— repository/           # 数据访问层
10 |   |   |       |— model/               # 实体类
11 |   |   |       |— config/              # 配置类
12 |   |   |— resources/
13 |   |       |— application.properties   # 配置文件
14 |   |       |— static/                  # 静态资源
15 |   |       |— templates/               # 模板文件
16 |   |— test/                            # 测试代码
17 |— pom.xml                             # Maven配置
18 |— README.md
```

5.2 @SpringBootApplication 注解

这是 Spring Boot 应用的**核心注解**，是一个组合注解：

```
1 @SpringBootApplication
2 public class MyAppApplication {
3     public static void main(String[] args) {
4         SpringApplication.run(MyAppApplication.class, args);
5     }
6 }
```

 Java

等价于三个注解

```
1 @Configuration           // 1. 配置类
2 @EnableAutoConfiguration // 2. 启用自动配置
3 @ComponentScan            // 3. 组件扫描
4 public class MyAppApplication {
```

 Java

```
5 // ...
6 }
```

各部分作用

注解	作用	说明
<code>@Configuration</code>	标识配置类	可以定义@Bean
<code>@EnableAutoConfiguration</code>	启用自动配置	根据依赖自动配置 Bean
<code>@ComponentScan</code>	组件扫描	扫描当前包及子包

5.3 常用注解速查

分层注解

1	<code>@RestController</code>	// REST控制器 (@Controller + @ResponseBody)	 Java
2	<code>@Service</code>	// 服务层	
3	<code>@Repository</code>	// 数据访问层	
4	<code>@Component</code>	// 通用组件	

依赖注入

1	<code>@Autowired</code>	// 自动注入 (字段注入)	 Java
2	<code>@Resource</code>	// JSR-250标准	
3	<code>@Inject</code>	// JSR-330标准	

TIP

推荐使用构造器注入而非字段注入，更易于测试。

配置相关

1	<code>@Configuration</code>	// 配置类	 Java
2	<code>@Bean</code>	// 定义Bean	
3	<code>@Value</code>	// 注入配置值	
4	<code>@ConfigurationProperties</code>	// 类型安全配置绑定	

6 内嵌 Web 容器与启动原理初探

6.1 内嵌 Web 容器

Spring Boot 默认内嵌 Tomcat 作为 Web 容器。

支持的容器

容器	Starter	特点
Tomcat	spring-boot-starter-tomcat	默认，成熟稳定

Jetty	spring-boot-starter-jetty	轻量，适合长连接
Undertow	spring-boot-starter-undertow	高性能，低内存

切换容器

```
1 <!-- 排除Tomcat, 使用Jetty -->
2 <dependency>
3   <groupId>org.springframework.boot</groupId>
4   <artifactId>spring-boot-starter-web</artifactId>
5   <exclusions>
6     <exclusion>
7       <groupId>org.springframework.boot</groupId>
8       <artifactId>spring-boot-starter-tomcat</artifactId>
9     </exclusion>
10  </exclusions>
11 </dependency>
12
13 <dependency>
14   <groupId>org.springframework.boot</groupId>
15   <artifactId>spring-boot-starter-jetty</artifactId>
16 </dependency>
```

6.2 启动流程简析

```
1 @SpringBootApplication
2 public class MyappApplication {
3     public static void main(String[] args) {
4         SpringApplication.run(MyappApplication.class, args);
5     }
6 }
```

主要步骤

1. 创建 SpringApplication 对象

- 推断应用类型 (SERVLET/REACTIVE/NONE)
- 加载 ApplicationContextInitializer
- 加载 ApplicationListener

2. 执行 run()方法

- 启动计时器
- 创建 ApplicationContext
- 刷新上下文 (加载 Bean)
- 调用 Runner (CommandLineRunner/ApplicationRunner)
- 返回 ApplicationContext

简化流程图

```
1 main()
2 ↓
3 SpringApplication构造
```

```
4   ↓
5   run()
6   ↓
7   创建Context
8   ↓
9   刷新Context (自动配置)
10  ↓
11  启动内嵌Tomcat
12  ↓
13  应用就绪
```

6.3 第一个 REST 接口

```
1  import org.springframework.web.bind.annotation.*;
2
3  @RestController
4  @RequestMapping("/api")
5  public class HelloController {
6
7      @GetMapping("/hello")
8      public String hello() {
9          return "Hello, Spring Boot!";
10     }
11
12     @GetMapping("/greet/{name}")
13     public String greet(@PathVariable String name) {
14         return "Hello, " + name + "!";
15     }
16 }
```

 Java

运行应用：

```
1 mvn spring-boot:run
```

 Bash

访问：

```
1 curl http://localhost:8080/api/hello
2 # 输出: Hello, Spring Boot!
3
4 curl http://localhost:8080/api/greet/World
5 # 输出: Hello, World!
```

 Bash

本章介绍了 Spring Boot 的基础知识，包括其诞生背景、核心价值、版本选型、环境准备、项目创建和启动原理。下一章将深入探讨 Spring Boot 的配置管理与自动配置机制。

Chapter 2

日志系统详解

日志是应用程序的“黑匣子”，记录系统运行状态。Spring Boot 的日志设计遵循门面模式，通过 SLF4J 统一接口，底层可以灵活切换实现框架，实现了日志系统的解耦和可扩展性。

1 Java 日志生态与 SLF4J 门面模式

1.1 Java 日志生态演变

在 SLF4J 出现之前，Java 日志领域存在多个竞争框架：

框架	发布年份	特点	问题
java.util.logging (JUL)	2002	JDK 自带	功能简单，性能一般
Log4j 1.x	2001	功能强大	已停止维护，有安全漏洞
Logback	2006	Log4j 改进版	需要 SLF4J 配合
Log4j 2.x	2014	高性能	API 不兼容 Log4j 1.x

存在的问题：

- 不同库使用不同的日志框架
- 项目中可能出现多个日志实现
- 配置复杂，难以统一管理

1.2 SLF4J 的解决方案

SLF4J (Simple Logging Facade for Java) 是一个日志门面 (Logging Facade)。

核心理念：

1 应用代码 → SLF4J API (接口) → 具体实现 (Logback/Log4j2/JUL)

优势：

1. 解耦：应用代码只依赖 SLF4J API
2. 灵活：可以随时切换底层实现
3. 统一：所有库都通过 SLF4J 输出日志
4. 性能：支持参数化日志，避免不必要的字符串拼接

```
1 // 使用SLF4J API (推荐)
2 import org.slf4j.Logger;
3 import org.slf4j.LoggerFactory;
4
```



```
5 public class UserService {
6     private static final Logger log = LoggerFactory.getLogger(UserService.class);
7
8     public void createUser(String name) {
9         // 参数化日志，只有当日志级别启用时才拼接字符串
10        log.info("Creating user: {}", name);
11    }
12 }
```

TIP

永远使用 SLF4J API，不要直接使用 Logback 或 Log4j 的 API。这样可以在不修改代码的情况下切换日志实现。

2 Spring Boot 默认日志框架

2.1 默认日志框架

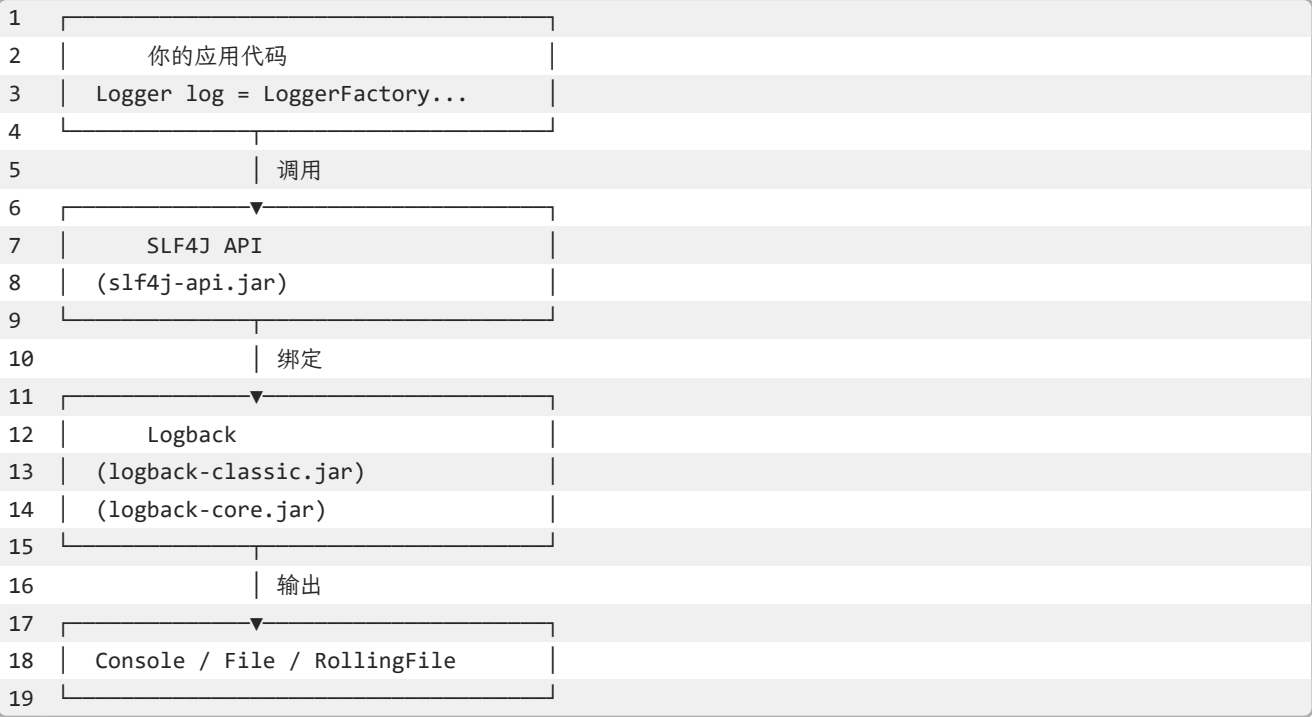
Spring Boot 默认使用 Logback 作为日志实现。

依赖关系：

```
1 <!-- spring-boot-starter -->
2 <dependency>
3     <groupId>org.springframework.boot</groupId>
4     <artifactId>spring-boot-starter</artifactId>
5 </dependency>
6     ↓ 传递依赖
7 <!-- spring-boot-starter-logging -->
8 <dependency>
9     <groupId>org.springframework.boot</groupId>
10    <artifactId>spring-boot-starter-logging</artifactId>
11 </dependency>
12    ↓ 包含
13 <!-- SLF4J API -->
14 <dependency>
15     <groupId>org.slf4j</groupId>
16     <artifactId>slf4j-api</artifactId>
17 </dependency>
18 <!-- Logback Classic (实现) -->
19 <dependency>
20     <groupId>ch.qos.logback</groupId>
21     <artifactId>logback-classic</artifactId>
22 </dependency>
23 <!-- Logback Core -->
24 <dependency>
25     <groupId>ch.qos.logback</groupId>
26     <artifactId>logback-core</artifactId>
27 </dependency>
28 <!-- JUL到SLF4J的桥接 -->
29 <dependency>
```

```
30 <groupId>org.slf4j</groupId>
31 <artifactId>jul-to-slf4j</artifactId>
32 </dependency>
```

架构图：



2.2 日志桥接 (Bridge)

Spring Boot 自动配置了日志桥接，将其他日志框架的输出重定向到 SLF4J：

桥接模块	作用	重定向
jcl-over-slf4j	Jakarta Commons Logging	→ SLF4J
jul-to-slf4j	java.util.logging	→ SLF4J
log4j-over-slf4j	Log4j 1.x	→ SLF4J

效果：即使第三方库使用 JUL 或 Log4j，日志也会统一通过 SLF4J 输出。

CAUTION

不要同时引入 `log4j-over-slf4j` 和 `slf4j-log4j12`，会导致循环依赖和 StackOverflowError。

3 Logback 配置详解

3.1 配置文件位置

Spring Boot 按以下顺序查找 Logback 配置：

- 1. `logback-spring.xml`（推荐，支持 Spring 特性）
- 2. `logback.xml`

3. application.properties/yml 中的配置

推荐：使用 logback-spring.xml，因为它支持 Spring 的 Profile 和属性占位符。

3.2 基本配置结构

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <configuration>
3
4      <!-- 属性定义 -->
5      <property name="LOG_PATTERN"
6              value="%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"/>
7      <property name="LOG_PATH" value="logs"/>
8
9      <!-- 控制台输出 -->
10     <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
11         <encoder>
12             <pattern>${LOG_PATTERN}</pattern>
13             <charset>UTF-8</charset>
14         </encoder>
15     </appender>
16
17     <!-- 文件输出 -->
18     <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
19         <file>${LOG_PATH}/application.log</file>
20         <encoder>
21             <pattern>${LOG_PATTERN}</pattern>
22         </encoder>
23         <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
24             <fileNamePattern>${LOG_PATH}/application.%d{yyyy-MM-dd}.log</fileNamePattern>
25             <maxHistory>30</maxHistory>
26             <totalSizeCap>1GB</totalSizeCap>
27         </rollingPolicy>
28     </appender>
29
30     <!-- 根日志器 -->
31     <root level="INFO">
32         <appender-ref ref="CONSOLE"/>
33         <appender-ref ref="FILE"/>
34     </root>
35
36     <!-- 特定包的日志级别 -->
37     <logger name="com.example.myapp" level="DEBUG"/>
38     <logger name="org.springframework.web" level="INFO"/>
39     <logger name="org.hibernate.SQL" level="DEBUG"/>
40
41 </configuration>
```

3.3 日志级别

级别	值	用途	生产环境
----	---	----	------

TRACE	0	最详细追踪	禁用
DEBUG	10	调试信息	禁用
INFO	20	重要信息	✓ 推荐
WARN	30	警告信息	✓ 推荐
ERROR	40	错误信息	✓ 推荐
OFF	Integer.MAX_VALUE	关闭日志	特殊场景

继承规则：

- 子 Logger 继承父 Logger 的级别
- 如果未显式设置，使用 root 的级别
- 更具体的配置覆盖通用配置

```
1 <!-- root设置为INFO -->
2 <root level="INFO">
3   <appender-ref ref="CONSOLE"/>
4 </root>
5
6 <!-- com.example包下的类使用DEBUG级别 -->
7 <logger name="com.example" level="DEBUG"/>
8
9 <!-- com.example.service包下的类使用TRACE级别 -->
10 <logger name="com.example.service" level="TRACE"/>
```

3.4 日志格式模式

常用模式字符：

模式	含义	示例
`%d{pattern}`	日期时间	`2024-01-15 10:30:45.123`
`%thread`	线程名	`main`, `http-nio-8080-exec-1`
`%-5level`	日志级别	`INFO`, `ERROR`
`%logger{length}`	Logger 名称	`c.e.m.UserService`
`%msg`	日志消息	`Creating user: Alice`
`%n`	换行符	
`%exception`	异常堆栈	完整堆栈信息

彩色输出（仅控制台）：

```
1 <pattern>%clr(%d{yyyy-MM-dd HH:mm:ss.SSS}){faint}
2   %clr(%5p) %clr(${PID:- }){magenta}
3   %clr(---){faint} %clr([%15.15t]){faint}
4   %clr(%-40.40logger{39}){cyan} %clr(:){faint} %m%n</pattern>
```

TIP

Spring Boot 默认的日志格式已经包含了彩色输出，开发时更易读。

3.5 application.properties 配置

对于简单场景，可以直接在配置文件中设置：

```
1  # 日志级别
2  logging.level.root=INFO
3  logging.level.com.example.myapp=DEBUG
4  logging.level.org.springframework.web=WARN
5
6  # 日志文件
7  logging.file.name=logs/application.log
8  logging.file.max-size=10MB
9  logging.file.max-history=30
10
11 # 日志格式
12 logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n
13 logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
14
15 # 日志组 (Spring Boot 2.1+)
16 logging.group.tomcat=org.apache.catalina,org.apache.coyote,org.apache.tomcat
17 logging.level.tomcat=DEBUG
```

NOTE

对于复杂的日志配置（如多 appender、滚动策略），建议使用 `logback-spring.xml`。

4 异步日志与性能优化

4.1 异步日志

异步日志可以提升性能，特别是高并发场景：

```
1  <appender name="ASYNC_FILE" class="ch.qos.logback.classic.AsyncAppender">
2      <!-- 队列大小 -->
3      <queueSize>512</queueSize>
4      <!-- 丢弃阈值（队列剩余容量低于此值时丢弃TRACE/DEBUG/INFO） -->
5      <discardingThreshold>0</discardingThreshold>
6      <!-- 是否包含调用者信息 -->
7      <includeCallerData>false</includeCallerData>
8
9      <appender-ref ref="FILE"/>
10 </appender>
11
12 <root level="INFO">
13     <appender-ref ref="ASYNC_FILE"/>
14 </root>
```

性能提升：

- 同步日志：约100K msg/s
- 异步日志：约1M msg/s

CAUTION

异步日志在应用关闭时可能丢失少量日志。设置 `queueSize` 时要权衡内存占用。

4.2 条件配置 (Profile)

`logback-spring.xml` 支持 Spring Profile:

```
1 <configuration>
2
3 <!-- 开发环境 -->
4 <springProfile name="dev">
5   <root level="DEBUG">
6     <appender-ref ref="CONSOLE"/>
7   </root>
8   <logger name="com.example" level="TRACE"/>
9 </springProfile>
10
11 <!-- 生产环境 -->
12 <springProfile name="prod">
13   <root level="WARN">
14     <appender-ref ref="ASYNC_FILE"/>
15   </root>
16   <logger name="com.example" level="INFO"/>
17 </springProfile>
18
19 <!-- 测试环境 -->
20 <springProfile name="test">
21   <root level="INFO">
22     <appender-ref ref="CONSOLE"/>
23   </root>
24 </springProfile>
25
26 </configuration>
```

5 结构化日志与 JSON 格式输出

5.1 为什么需要结构化日志

传统日志的问题:

```
1 2024-01-15 10:30:45.123 [http-nio-8080-exec-1] INFO c.e.m.UserService - Creating user: Alice,
  age=25, email=alice@example.com
```

- 难以解析和查询
- 无法高效聚合分析
- 不支持多维度的过滤

结构化日志:

```
1 {
2   "timestamp": "2024-01-15T10:30:45.123Z",
```

```
3  "level": "INFO",
4  "thread": "http-nio-8080-exec-1",
5  "logger": "com.example.myapp.UserService",
6  "message": "Creating user",
7  "user_name": "Alice",
8  "user_age": 25,
9  "user_email": "alice@example.com",
10 "trace_id": "abc123",
11 "span_id": "def456"
12 }
```

优势:

- 易于机器解析
- 支持复杂查询
- 便于日志聚合（ELK、Loki）
- 支持链路追踪

TIP

微服务架构中，结构化日志是标配，强烈推荐使用。

5.2 JSON 格式化日志

使用 logstash-logback-encoder

```
1 <dependency>
2   <groupId>net.logstash.logback</groupId>
3   <artifactId>logstash-logback-encoder</artifactId>
4   <version>7.4</version>
5 </dependency>
```

配置:

```
1 <appender name="JSON_CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
2   <encoder class="net.logstash.logback.encoder.LogstashEncoder">
3     <!-- 自定义字段 -->
4     <customFields>{"app": "myapp", "env": "${APP_ENV:-dev}"}</customFields>
5
6     <!-- 包含MDC -->
7     <includeMdcKeyName>trace_id</includeMdcKeyName>
8     <includeMdcKeyName>user_id</includeMdcKeyName>
9
10    <!-- 时间戳格式 -->
11    <timestampPattern>yyyy-MM-dd'T'HH:mm:ss.SSS'Z'</timestampPattern>
12    <timeZone>UTC</timeZone>
13  </encoder>
14 </appender>
15
16 <root level="INFO">
17   <appender-ref ref="JSON_CONSOLE"/>
18 </root>
```

输出:

```
1  {
2    "@timestamp": "2024-01-15T10:30:45.123Z",
3    "@version": "1",
4    "message": "Creating order",
5    "logger_name": "com.example.myapp.OrderService",
6    "thread_name": "http-nio-8080-exec-1",
7    "level": "INFO",
8    "level_value": 20000,
9    "app": "myapp",
10   "env": "dev",
11   "trace_id": "abc123def456",
12   "user_id": "user123"
13 }
```

6 MDC 与链路追踪上下文

6.1 MDC (Mapped Diagnostic Context)

MDC 是 SLF4J 提供的线程级别的上下文存储，用于在日志中添加额外信息。

基本原理

1 请求进入 → 生成trace_id → 放入MDC → 所有日志自动包含trace_id → 请求结束清除MDC

text

基本用法

```
1  import org.slf4j.MDC;
2
3  @Service
4  public class OrderService {
5      private static final Logger log = LoggerFactory.getLogger(OrderService.class);
6
7      public void createOrder(String orderId) {
8          // 将订单ID放入MDC
9          MDC.put("order_id", orderId);
10         MDC.put("user_id", "user123");
11
12         try {
13             log.info("Creating order");
14             // ... 业务逻辑
15             log.info("Order created successfully");
16         } finally {
17             // 清理MDC (重要!)
18             MDC.clear();
19         }
20     }
21 }
```

日志输出:

```
1 2024-01-15 10:30:45.123 [http-nio-8080-exec-1] INFO c.e.m.OrderService - Creating order
  {order_id=ORD001, user_id=user123}
2 2024-01-15 10:30:45.456 [http-nio-8080-exec-1] INFO c.e.m.OrderService - Order created
  successfully {order_id=ORD001, user_id=user123}
```

CAUTION

必须在 finally 块中清理 MDC，否则在线程池环境下会导致内存泄漏和数据污染。

在 Filter 中设置 Trace ID

```
1 import org.slf4j.MDC;
2 import org.springframework.stereotype.Component;
3 import javax.servlet.*;
4 import java.io.IOException;
5 import java.util.UUID;
6
7 @Component
8 public class TraceIdFilter implements Filter {
9
10     private static final String TRACE_ID_KEY = "trace_id";
11
12     @Override
13     public void doFilter(ServletRequest request, ServletResponse response,
14                         FilterChain chain) throws IOException, ServletException {
15         try {
16             // 从请求头获取或生成新的trace_id
17             String traceId = ((HttpServletRequest) request).getHeader("X-Trace-Id");
18             if (traceId == null || traceId.isEmpty()) {
19                 traceId = UUID.randomUUID().toString().replace("-", "");
20             }
21
22             // 放入MDC
23             MDC.put(TRACE_ID_KEY, traceId);
24
25             // 继续处理
26             chain.doFilter(request, response);
27         } finally {
28             // 清理MDC
29             MDC.clear();
30         }
31     }
32 }
```

配置 Logback pattern:

```
1 <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %X{trace_id} -
  %msg%n</pattern>
```

效果:

```
1 2024-01-15 10:30:45.123 [http-nio-8080-exec-1] INFO c.e.m.OrderService - abc123def456 - Creating
  order
2 2024-01-15 10:30:45.456 [http-nio-8080-exec-1] INFO c.e.m.PaymentService - abc123def456 -
  Processing payment
```

TIP

通过 trace_id 可以追踪一个请求在所有服务中的完整调用链路。

7 敏感信息脱敏与安全日志

7.1 为什么需要脱敏

生产环境必须对敏感信息（手机号、身份证、银行卡等）进行脱敏，符合 GDPR 等法规要求。

7.2 实现方式

```
1 import ch.qos.logback.classic.pattern.MessageConverter;
2 import ch.qos.logback.classic.spi.ILoggingEvent;
3
4 public class SensitiveDataConverter extends MessageConverter {
5
6     @Override
7     public String convert(ILoggingEvent event) {
8         String message = event.getFormattedMessage();
9
10        // 脱敏手机号
11        message = message.replaceAll("(\\d{3})\\d{4}(\\d{4})", "$1****$2");
12
13        // 脱敏邮箱
14        message = message.replaceAll("(\\w{3})\\w*@", "$1***@");
15
16        // 脱敏身份证
17        message = message.replaceAll("(\\d{6})\\d{8}(\\d{4})", "$1*****$2");
18
19        return message;
20    }
21 }
```

```
1 <conversionRule conversionWord="msg"
2     converterClass="com.example.SensitiveDataConverter" />
```

NOTE

生产环境必须对敏感信息（手机号、身份证、银行卡等）进行脱敏，符合 GDPR 等法规要求。

8 多环境日志配置

8.1 基于 Profile 的配置

logback-spring.xml 支持 Spring Profile，可以为不同环境配置不同的日志策略：

```
1 <configuration>
2
```

```
3      <!-- 开发环境 -->
4      <springProfile name="dev">
5          <root level="DEBUG">
6              <appender-ref ref="CONSOLE"/>
7          </root>
8          <logger name="com.example" level="TRACE"/>
9      </springProfile>
10
11     <!-- 测试环境 -->
12     <springProfile name="test">
13         <root level="INFO">
14             <appender-ref ref="CONSOLE"/>
15             <appender-ref ref="FILE"/>
16         </root>
17     </springProfile>
18
19     <!-- 生产环境 -->
20     <springProfile name="prod">
21         <root level="WARN">
22             <appender-ref ref="ASYNC_FILE"/>
23         </root>
24         <logger name="com.example" level="INFO"/>
25     </springProfile>
26
27 </configuration>
```

8.2 动态调整日志级别

Spring Boot Actuator 提供了动态调整日志级别的端点：

```
1 # 查看当前日志级别
2 curl http://localhost:8080/actuator/loggers
3
4 # 修改某个包的日志级别
5 curl -X POST http://localhost:8080/actuator/loggers/com.example \
6     -H "Content-Type: application/json" \
7     -d '{"configuredLevel": "DEBUG"}'
```

TIP

生产环境可以通过 Actuator 动态调整日志级别，无需重启应用。

9 自定义 Appender 与日志扩展

9.1 自定义 Appender

可以将日志发送到自定义目标（如 Kafka、Elasticsearch 等）：

```
1 import ch.qos.logback.core.AppenderBase;
2
```

```
3 public class KafkaAppender extends AppenderBase<ILoggingEvent> {
4     private KafkaProducer<String, String> producer;
5     private String topic;
6
7     @Override
8     public void start() {
9         // 初始化Kafka Producer
10        producer = new KafkaProducer<>(...);
11        super.start();
12    }
13
14    @Override
15    protected void append(ILoggingEvent event) {
16        String message = String.format("[%s] %s - %s",
17            event.getLevel(),
18            event.getLoggerName(),
19            event.getFormattedMessage()
20        );
21        producer.send(new ProducerRecord<>(topic, message));
22    }
23
24    @Override
25    public void stop() {
26        producer.close();
27        super.stop();
28    }
29 }
```

```
1 <appender name="KAFKA" class="com.example.KafkaAppender">
2     <topic>application-logs</topic>
3 </appender>
4
5 <root level="INFO">
6     <appender-ref ref="KAFKA"/>
7 </root>
```

xml

9.2 最佳实践总结

1. 统一的日志工具类

```
1 import org.slf4j.Logger;
2 import org.slf4j.LoggerFactory;
3 import org.slf4j.MDC;
4
5 public class LogUtils {
6
7     public static void putTraceId(String traceId) {
8         MDC.put("trace_id", traceId);
9     }
10
11     public static void putUserId(String userId) {
12         MDC.put("user_id", userId);
```

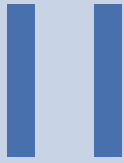
 Java

```
13     }
14
15     public static void putRequestId(String requestId) {
16         MDC.put("request_id", requestId);
17     }
18
19     public static void clear() {
20         MDC.clear();
21     }
22
23     // 便捷方法：记录业务操作
24     public static void logBusinessOperation(Logger log, String operation,
25                                           String status, Object... params) {
26         log.info("operation={}, status={}, params={}", operation, status, params);
27     }
28 }
```

2. AOP 自动添加上下文

```
1  import org.aspectj.lang.ProceedingJoinPoint;
2  import org.aspectj.lang.annotation.Around;
3  import org.aspectj.lang.annotation.Aspect;
4  import org.slf4j.MDC;
5  import org.springframework.stereotype.Component;
6  import java.util.UUID;
7
8  @Aspect
9  @Component
10 public class LoggingAspect {
11
12     @Around("@annotation(org.springframework.web.bind.annotation.RestController)")
13     public Object addLoggingContext(ProceedingJoinPoint joinPoint) throws Throwable {
14         String traceId = UUID.randomUUID().toString().replace("-", "");
15         MDC.put("trace_id", traceId);
16
17         try {
18             return joinPoint.proceed();
19         } finally {
20             MDC.clear();
21         }
22     }
23 }
```

本章详细介绍了 Spring Boot 的日志系统，从 SLF4J 门面模式到 Logback 配置，从异步优化到结构化日志，为构建可观测性系统打下基础。下一章将深入探讨监控与可观测性的其他两大支柱：指标和链路追踪。



Spring Boot 核心

3.	定时任务与异步处理 ·····	34
4.	日志系统详解 ·····	68
5.	监控与可观测性 ·····	82

Chapter 3

定时任务与异步处理

NOTE

本文档基于 Spring Boot 3.2.4 和 Java 17 编写，所有代码示例均在该环境下测试通过。

本章将详细介绍 Spring Boot 中的定时任务和异步处理机制，包括：

- 基础定时任务配置和使用
- 异步方法执行和线程池管理
- 高级定时任务（动态、分布式）
- Quartz 框架集成
- 异步编程最佳实践

TIP

建议按顺序阅读，从基础到高级逐步深入。每个章节都有完整的代码示例，可以直接在项目中使用。

1 定时任务基础

1.1 @Scheduled 注解详解

Spring Boot 提供了 `@Scheduled` 注解来简化定时任务的配置。该注解可以标注在方法上，使该方法按照指定的时间规则周期性执行。

NOTE

使用 `@Scheduled` 前，需要在启动类或配置类上添加 `@EnableScheduling` 注解以启用定时任务支持。

基本使用示例：

```
1 import org.springframework.scheduling.annotation.Scheduled;
2 import org.springframework.stereotype.Component;
3 import java.time.LocalDateTime;
4
5 @Component
6 public class SimpleTask {
7
8     @Scheduled(fixedRate = 5000)
9     public void executeFixedRate() {
10         System.out.println("每5秒执行一次: " + LocalDateTime.now());
11     }
12 }
```



TIP

`fixedRate` 表示从上一次任务开始执行的时间点计算间隔，而 `fixedDelay` 表示从上一次任务结束的时间点计算间隔。

1.2 Cron 表达式语法

Cron 表达式是定时任务中最灵活的调度方式，由 6 或 7 个字段组成：

位置	字段	允许值	特殊字符
1	秒	0-59	, - * /
2	分	0-59	, - * /
3	时	0-23	, - * /
4	日	1-31	, - * ? / L W
5	月	1-12 或 JAN-DEC	, - * /
6	周	0-7 或 SUN-SAT	, - * ? / L #
7	年（可选）	1970-2099	, - * /

CAUTION

「日」和「周」字段不能同时指定具体值，其中一个必须为 `?`（不指定）。

常用 Cron 表达式示例：

表达式	含义
<code>"0 0 * * * *"</code>	每小时整点执行
<code>"0 0 0 * * *"</code>	每天凌晨执行
<code>"0 */5 * * * *"</code>	每 5 分钟执行一次
<code>"0 0 9-17 * * MON-FRI"</code>	工作日 9 点到 17 点每小时执行
<code>"0 0 12 1 * ?"</code>	每月 1 号中午 12 点执行
<code>"0 0/30 8-10 * * *"</code>	每天 8 点到 10 点，每 30 分钟执行

实际应用示例：

```
1 import org.springframework.scheduling.annotation.Scheduled;
2 import org.springframework.stereotype.Component;
3 import java.time.LocalDateTime;
4
5 @Component
6 public class CronTask {
7
8     // 每天凌晨2点执行数据备份
9     @Scheduled(cron = "0 0 2 * * ?")
10    public void backupData() {
11        System.out.println("执行数据备份: " + LocalDateTime.now());
12    }
13 }
```



```
14 // 每周一上午9点发送周报
15 @Scheduled(cron = "0 0 9 ? * MON")
16 public void sendWeeklyReport() {
17     System.out.println("发送周报: " + LocalDateTime.now());
18 }
19 }
```

TIP

可以使用在线工具如 <https://crontab.guru/> 来验证和生成 Cron 表达式。

1.3 固定速率与固定延迟

`@Scheduled` 提供了三种主要的调度策略：

fixedRate（固定速率）

从上一次任务开始执行的时间点计算下一次执行的间隔。

```
1 import org.springframework.scheduling.annotation.Scheduled;
2 import java.util.Date;
3
4 @Scheduled(fixedRate = 3000)
5 public void fixedRateTask() {
6     long start = System.currentTimeMillis();
7     System.out.println("任务开始: " + new Date());
8
9     // 模拟耗时操作
10    try {
11        Thread.sleep(2000);
12    } catch (InterruptedException e) {
13        e.printStackTrace();
14    }
15
16    long end = System.currentTimeMillis();
17    System.out.println("任务结束, 耗时: " + (end - start) + "ms");
18 }
```

INFO

如果任务执行时间超过 `fixedRate` 设定的间隔，下一次任务会立即执行，不会等待。

fixedDelay（固定延迟）

从上一次任务执行完成的时间点计算下一次执行的间隔。

```
1 import org.springframework.scheduling.annotation.Scheduled;
2 import java.util.Date;
3
4 @Scheduled(fixedDelay = 3000)
5 public void fixedDelayTask() {
6     long start = System.currentTimeMillis();
7     System.out.println("任务开始: " + new Date());
8 }
```

```
9    // 模拟耗时操作
10   try {
11       Thread.sleep(2000);
12   } catch (InterruptedException e) {
13       e.printStackTrace();
14   }
15
16   long end = System.currentTimeMillis();
17   System.out.println("任务结束, 耗时: " + (end - start) + "ms");
18 }
```

NOTE

`fixedDelay` 保证了任务之间的间隔时间是固定的, 适合需要严格控制执行频率的场景。

initialDelay (初始延迟)

应用启动后, 延迟指定时间再开始执行定时任务。

```
1 import org.springframework.scheduling.annotation.Scheduled;
2 import java.util.Date;
3
4 // 应用启动后延迟10秒, 然后每5秒执行一次
5 @Scheduled(fixedRate = 5000, initialDelay = 10000)
6 public void delayedTask() {
7     System.out.println("延迟任务执行: " + new Date());
8 }
```

 Java

特性	fixedRate	fixedDelay
计算起点	上次任务开始时间	上次任务结束时间
任务重叠	可能发生	不会发生
适用场景	周期性数据采集	资源密集型任务
稳定性	可能累积延迟	间隔稳定

WARNING

默认情况下, Spring 的定时任务是单线程串行执行的。如果某个任务执行时间过长, 会阻塞后续任务的执行。建议使用线程池来解决这个问题 (后续章节会详细介绍)。

2 异步处理核心

2.1 @Async 注解使用

Spring Boot 提供了 `@Async` 注解来实现方法的异步执行。标注了该注解的方法会在独立的线程中运行, 不会阻塞调用线程。

NOTE

使用 `@Async` 前, 需要在启动类或配置类上添加 `@EnableAsync` 注解以启用异步支持。

基本使用示例:

```
1 import org.springframework.scheduling.annotation.Async;
2 import org.springframework.stereotype.Service;
3 import java.util.concurrent.CompletableFuture;
4
5 @Service
6 public class AsyncService {
7
8     @Async
9     public void asyncMethod() {
10         System.out.println("异步方法执行, 线程: " + Thread.currentThread().getName());
11         // 模拟耗时操作
12         try {
13             Thread.sleep(3000);
14         } catch (InterruptedException e) {
15             e.printStackTrace();
16         }
17         System.out.println("异步方法完成");
18     }
19 }
```

调用异步方法:

```
1 @RestController
2 public class AsyncController {
3
4     @Autowired
5     private AsyncService asyncService;
6
7     @GetMapping("/async")
8     public String handleAsync() {
9         System.out.println("主线程开始: " + Thread.currentThread().getName());
10
11         // 异步方法立即返回, 不阻塞
12         asyncService.asyncMethod();
13
14         System.out.println("主线程结束: " + Thread.currentThread().getName());
15         return "请求已接受, 异步任务正在执行";
16     }
17 }
```

TIP

`@Async` 可以标注在类上, 这样该类的所有 `public` 方法都会异步执行。

CAUTION

`@Async` 方法必须是 `public` 的, 且在同一个类内部调用时不会生效 (因为 Spring AOP 代理机制的限制)。

2.2 线程池配置与管理

默认的异步执行器使用 `SimpleAsyncTaskExecutor`, 它每次调用都会创建新线程, 不适合生产环境。建议配置自定义线程池。

基础线程池配置

```
1 import org.springframework.context.annotation.Bean;
2 import org.springframework.context.annotation.Configuration;
3 import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;
4 import java.util.concurrent.Executor;
5 import java.util.concurrent.ThreadPoolExecutor;
6
7 @Configuration
8 public class AsyncConfig {
9
10     @Bean("taskExecutor")
11     public Executor taskExecutor() {
12         ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
13
14         // 核心线程数：线程池创建时的初始线程数
15         executor.setCorePoolSize(5);
16
17         // 最大线程数：线程池允许的最大线程数
18         executor.setMaxPoolSize(10);
19
20         // 队列容量：当核心线程都在忙时，新任务放入队列等待
21         executor.setQueueCapacity(100);
22
23         // 线程名称前缀：便于调试和监控
24         executor.setThreadNamePrefix("async-task-");
25
26         // 拒绝策略：当队列满且达到最大线程数时的处理方式
27         executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
28
29         // 等待所有任务结束后再关闭线程池
30         executor.setWaitForTasksToCompleteOnShutdown(true);
31
32         // 等待时间（秒）
33         executor.setAwaitTerminationSeconds(60);
34
35         executor.initialize();
36         return executor;
37     }
38 }
```

参数	说明	推荐值
corePoolSize	核心线程数	CPU 核数 × 2
maxPoolSize	最大线程数	CPU 核数 × 4
queueCapacity	队列容量	根据业务量设定，如 100-1000
keepAliveSeconds	空闲线程存活时间	60 秒
rejectedExecutionHandler	拒绝策略	CallerRunsPolicy 或 AbortPolicy

指定线程池

可以在 `@Async` 注解中指定使用的线程池 Bean 名称：

```
1 @Service
2 public class MultiPoolService {
3
4     // 使用默认的 taskExecutor
5     @Async
6     public void defaultAsyncTask() {
7         System.out.println("默认线程池: " + Thread.currentThread().getName());
8     }
9
10    // 使用指定的线程池
11    @Async("taskExecutor")
12    public void customAsyncTask() {
13        System.out.println("自定义线程池: " + Thread.currentThread().getName());
14    }
15 }
```

多线程池配置

对于不同优先级的任务，可以配置多个线程池：

```
1 @Configuration
2 public class MultiExecutorConfig {
3
4     // 高优先级任务线程池
5     @Bean("highPriorityExecutor")
6     public Executor highPriorityExecutor() {
7         ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
8         executor.setCorePoolSize(10);
9         executor.setMaxPoolSize(20);
10        executor.setQueueCapacity(50);
11        executor.setThreadNamePrefix("high-priority-");
12        executor.initialize();
13        return executor;
14    }
15
16    // 低优先级任务线程池
17    @Bean("lowPriorityExecutor")
18    public Executor lowPriorityExecutor() {
19        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
20        executor.setCorePoolSize(2);
21        executor.setMaxPoolSize(5);
22        executor.setQueueCapacity(200);
23        executor.setThreadNamePrefix("low-priority-");
24        executor.initialize();
25        return executor;
26    }
27 }
```


WARNING

线程池参数需要根据实际业务场景调优。核心线程数过多会导致上下文切换开销增加，过少则无法充分利用 CPU 资源。

2.3 异步返回值 Future

`@Async` 方法可以返回 `Future`、`CompletableFuture` 或 `ListenableFuture`，以便获取异步执行的结果。

CompletableFuture 返回类型

`CompletableFuture` 是 Java 8 引入的强大异步编程工具：

```
1  @Service Java
2  public class AsyncResultService {
3
4      @Async
5      public CompletableFuture<String> asyncWithResult() {
6          System.out.println("异步任务开始: " + Thread.currentThread().getName());
7
8          try {
9              // 模拟耗时操作
10             Thread.sleep(2000);
11         } catch (InterruptedException e) {
12             e.printStackTrace();
13         }
14
15         String result = "异步任务完成";
16         System.out.println("异步任务结束");
17
18         return CompletableFuture.completedFuture(result);
19     }
20
21     @Async
22     public CompletableFuture<Integer> asyncCalculation(int a, int b) {
23         try {
24             Thread.sleep(1000);
25         } catch (InterruptedException e) {
26             e.printStackTrace();
27         }
28
29         return CompletableFuture.completedFuture(a + b);
30     }
31 }
```

调用并获取结果：

```
1  @RestController Java
2  public class AsyncResultController {
3
4      @Autowired
5      private AsyncResultService asyncResultService;
```

```
6
7     @GetMapping("/result")
8     public String handleAsyncResult() throws Exception {
9         System.out.println("主线程开始");
10
11         // 发起异步调用
12         CompletableFuture<String> future = asyncResultService.asyncWithResult();
13
14         // 可以继续执行其他逻辑
15         System.out.println("主线程继续执行其他任务...");
16
17         // 阻塞等待结果（超时时间5秒）
18         String result = future.get(5, TimeUnit.SECONDS);
19
20         System.out.println("获取到结果: " + result);
21         return "结果: " + result;
22     }
23 }
```

多个异步任务组合

`CompletableFuture` 支持多个异步任务的组合:

```
1  @Service Java
2  public class CombinedAsyncService {
3
4      @Autowired
5      private AsyncResultService asyncResultService;
6
7      public String executeParallel() throws Exception {
8          // 并行执行多个异步任务
9          CompletableFuture<String> task1 = asyncResultService.asyncWithResult();
10         CompletableFuture<Integer> task2 = asyncResultService.asyncCalculation(10, 20);
11
12         // 等待所有任务完成
13         CompletableFuture.allOf(task1, task2).join();
14
15         // 获取结果
16         String result1 = task1.get();
17         Integer result2 = task2.get();
18
19         return result1 + ", 计算结果: " + result2;
20     }
21
22     // 链式调用
23     public CompletableFuture<String> chainedAsync() {
24         return asyncResultService.asyncWithResult()
25             .thenApply(result -> result.toUpperCase())
26             .thenApply(result -> "处理后: " + result)
27             .exceptionally(ex -> "异常: " + ex.getMessage());
28     }
29 }
```

方法	用途	示例
allOf	等待所有任务完成	CompletableFuture.allOf(f1, f2, f3)
anyOf	任一任务完成即返回	CompletableFuture.anyOf(f1, f2)
thenApply	转换结果	future.thenApply(s -> s.toUpperCase())
thenAccept	消费结果	future.thenAccept(System.out::println)
thenCompose	链式异步调用	future.thenCompose(this::anotherAsync)
exceptionally	异常处理	future.exceptionally(ex -> "default")

INFO

使用 `CompletableFuture` 可以实现复杂的异步编排，避免回调地狱（Callback Hell），使代码更加清晰易读。

异常处理

异步方法的异常不会直接抛出给调用者，需要特殊处理：

```
1 @Service
2 public class AsyncExceptionHandler {
3
4     @Async
5     public CompletableFuture<String> asyncWithException() {
6         try {
7             // 可能抛出异常的操作
8             if (Math.random() > 0.5) {
9                 throw new RuntimeException("随机异常");
10            }
11            return CompletableFuture.completedFuture("成功");
12        } catch (Exception e) {
13            // 返回异常结果的 CompletableFuture
14            CompletableFuture<String> future = new CompletableFuture<>();
15            future.completeExceptionally(e);
16            return future;
17        }
18    }
19 }
```

调用时处理异常：

```
1 @GetMapping("/exception")
2 public String handleException() {
3     CompletableFuture<String> future = asyncExceptionHandler.asyncWithException();
4
5     return future
6         .thenApply(result -> "结果: " + result)
7         .exceptionally(ex -> "捕获异常: " + ex.getCause().getMessage())
8         .join();
9 }
```

DANGER

异步方法中的未捕获异常会被线程池的异常处理器吞掉，不会传播到调用线程。务必做好异常处理和日志记录。

3 高级定时任务

3.1 动态定时任务

在实际业务中，经常需要动态调整定时任务的执行时间或启停状态，而不是在代码中硬编码 Cron 表达式。

基于数据库的动态配置

将定时任务的配置存储在数据库中，通过管理界面动态修改：

```
1  import org.springframework.scheduling.Trigger;
2  import org.springframework.scheduling.TriggerContext;
3  import org.springframework.scheduling.annotation.SchedulingConfigurer;
4  import org.springframework.scheduling.config.ScheduledTaskRegistrar;
5  import org.springframework.scheduling.support.CronTrigger;
6  import org.springframework.stereotype.Component;
7
8  @Component
9  public class DynamicScheduledTask implements SchedulingConfigurer {
10
11      @Autowired
12      private TaskConfigRepository taskConfigRepository;
13
14      @Override
15      public void configureTasks(ScheduledTaskRegistrar taskRegistrar) {
16          taskRegistrar.addTriggerTask(
17              // 任务逻辑
18              () -> {
19                  System.out.println("执行动态定时任务: " + LocalDateTime.now());
20                  // 执行业务逻辑
21              },
22              // 触发器：从数据库读取 Cron 表达式
23              new Trigger() {
24                  @Override
25                  public Date nextExecutionTime(TriggerContext triggerContext) {
26                      // 从数据库获取最新的 Cron 表达式
27                      String cron = taskConfigRepository.findCronByTaskName("dynamicTask");
28                      if (cron == null || cron.isEmpty()) {
29                          return null; // 任务暂停
30                      }
31                      CronTrigger trigger = new CronTrigger(cron);
32                      return trigger.nextExecutionTime(triggerContext);
33                  }
34              }
35          );
36      }
37  }
```

```
36     }  
37 }
```

数据库表设计示例：

字段	类型	说明
id	BIGINT	主键
task_name	VARCHAR(100)	任务名称（唯一）
cron_expression	VARCHAR(50)	Cron 表达式
enabled	BOOLEAN	是否启用
description	VARCHAR(500)	任务描述
update_time	DATETIME	最后更新时间

使用 TaskScheduler 动态注册

通过 `TaskScheduler` 可以在运行时动态创建和取消定时任务：

```
1  import org.springframework.scheduling.TaskScheduler;  
2  import org.springframework.scheduling.support.CronTrigger;  
3  import org.springframework.stereotype.Service;  
4  import java.util.concurrent.ScheduledFuture;  
5  import java.util.Map;  
6  import java.util.concurrent.ConcurrentHashMap;  
7  
8  @Service  
9  public class DynamicTaskService {  
10  
11      @Autowired  
12      private TaskScheduler taskScheduler;  
13  
14      // 存储已调度的任务  
15      private Map<String, ScheduledFuture<?>> scheduledTasks = new ConcurrentHashMap<>();  
16  
17      /**  
18       * 动态添加定时任务  
19       */  
20      public void addTask(String taskId, Runnable task, String cronExpression) {  
21          // 如果任务已存在，先取消  
22          cancelTask(taskId);  
23  
24          // 注册新任务  
25          ScheduledFuture<?> future = taskScheduler.schedule(  
26              task,  
27              new CronTrigger(cronExpression)  
28          );  
29  
30          scheduledTasks.put(taskId, future);  
31          System.out.println("任务 " + taskId + " 已注册, Cron: " + cronExpression);  
32      }  
33  }
```

```
34  /**
35   * 取消定时任务
36   */
37  public void cancelTask(String taskId) {
38      ScheduledFuture<?> future = scheduledTasks.remove(taskId);
39      if (future != null) {
40          future.cancel(false);
41          System.out.println("任务 " + taskId + " 已取消");
42      }
43  }
44
45  /**
46   * 更新任务的 Cron 表达式
47   */
48  public void updateTaskCron(String taskId, String newCron) {
49      // 先取消旧任务
50      cancelTask(taskId);
51
52      // 重新注册（需要重新传入任务逻辑）
53      // 实际应用中可以从数据库或其他存储中获取任务逻辑
54      System.out.println("任务 " + taskId + " 的 Cron 已更新为: " + newCron);
55  }
56 }
```

TIP

动态定时任务适合场景：用户自定义报表生成时间、灵活的活动提醒、可配置的清理策略等。

WARNING

动态任务需要注意线程安全问题，建议使用 `ConcurrentHashMap` 存储任务引用，并在操作时做好同步控制。

3.2 分布式定时任务

在微服务架构或多实例部署的场景下，同一个定时任务可能在多个节点上同时执行，导致数据重复处理等问题。

常见问题

问题	说明	影响
重复执行	多实例同时执行同一任务	数据重复、资源浪费
单点故障	某个实例宕机导致任务不执行	业务中断
负载不均	某些实例执行过多任务	性能瓶颈

解决方案对比

方案	优点	缺点	适用场景
数据库锁	实现简单	性能较差，有锁竞争	小规模应用
Redis 分布式锁	性能好，实现相对简单	需要 Redis 依赖	中等规模应用

Quartz 集群	功能完善，支持持久化	配置复杂，重量级	大型企业应用
XXL-JOB	可视化界面，功能强大	需要额外部署调度中心	微服务架构
Elastic-Job	去中心化，高可用	依赖 ZooKeeper	高并发场景

基于 Redis 的分布式锁实现

步骤一：添加依赖

在 pom.xml 中添加 Spring Data Redis：

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-data-redis</artifactId>
4 </dependency>
```

NOTE

Spring Boot 3.2.4 默认使用 Lettuce 作为 Redis 客户端。如果需要，也可以切换到 Jedis。

步骤二：配置 Redis 连接

在 application.yml 中配置：

```
1 spring:
2   data:
3     redis:
4       host: localhost
5       port: 6379
6       password: your_password # 如果有密码
7       database: 0
8       lettuce:
9         pool:
10          max-active: 8
11          max-idle: 8
12          min-idle: 0
```

步骤三：实现分布式锁服务

```
1 import org.springframework.data.redis.core.StringRedisTemplate;
2 import org.springframework.data.redis.core.script.DefaultRedisScript;
3 import org.springframework.stereotype.Component;
4 import java.util.Collections;
5 import java.util.concurrent.TimeUnit;
6
7 @Component
8 public class DistributedLockService {
9
10   @Autowired
11   private StringRedisTemplate redisTemplate;
12
13   /**
```

```
14      * 尝试获取分布式锁
15      * @param lockKey 锁的 key
16      * @param requestId 请求标识 (用于释放锁时验证)
17      * @param expireTime 锁的过期时间 (秒)
18      * @return 是否获取成功
19      */
20      public boolean tryLock(String lockKey, String requestId, long expireTime) {
21          Boolean result = redisTemplate.opsForValue()
22              .setIfAbsent(lockKey, requestId, expireTime, TimeUnit.SECONDS);
23          return result != null && result;
24      }
25
26      /**
27      * 释放分布式锁
28      * @param lockKey 锁的 key
29      * @param requestId 请求标识
30      * @return 是否释放成功
31      */
32      public boolean releaseLock(String lockKey, String requestId) {
33          String script = "if redis.call('get', KEYS[1]) == ARGV[1] then " +
34              "return redis.call('del', KEYS[1]) else return 0 end";
35          Long result = redisTemplate.execute(
36              new DefaultRedisScript<>(script, Long.class),
37              Collections.singletonList(lockKey),
38              requestId
39          );
40          return result != null && result == 1;
41      }
42  }
```

在定时任务中使用分布式锁：

```
1  @Component Java
2  public class DistributedScheduledTask {
3
4      @Autowired
5      private DistributedLockService lockService;
6
7      @Scheduled(cron = "0 0 2 * * ?")
8      public void distributedTask() {
9          String lockKey = "lock:daily-report-task";
10         String requestId = UUID.randomUUID().toString();
11
12         try {
13             // 尝试获取锁，过期时间设置为任务预计执行时间的 2 倍
14             boolean locked = lockService.tryLock(lockKey, requestId, 600);
15
16             if (!locked) {
17                 System.out.println("其他实例正在执行，跳过本次任务");
18                 return;
19             }
20
21             System.out.println("获取锁成功，开始执行任务");
```



```
22
23         // 执行业务逻辑
24         executeBusinessLogic();
25
26     } catch (Exception e) {
27         log.error("定时任务执行失败", e);
28     } finally {
29         // 确保释放锁
30         lockService.releaseLock(lockKey, requestId);
31         System.out.println("锁已释放");
32     }
33 }
34
35 private void executeBusinessLogic() {
36     // 业务逻辑
37 }
38 }
```

NOTE

使用 Redis 分布式锁时，务必设置合理的过期时间，防止死锁。同时，释放锁时要验证锁的持有者，避免误删其他线程的锁。

XXL-JOB 集成

XXL-JOB 是一个轻量级分布式任务调度平台，提供了可视化的管理界面。

核心特性：

- 可视化任务管理
- 弹性扩容缩容
- 故障转移和失败重试
- 分片广播任务
- 多种路由策略
- 实时日志查看

步骤一：部署调度中心

下载并启动 xxl-job-admin（调度中心）：

```
1 # 从 GitHub 下载
2 git clone https://github.com/xuxueli/xxl-job.git
3 cd xxl-job
4
5 # 执行数据库脚本 (tables_xx1_job.sql)
6 mysql -u root -p < doc/db/tables_xx1_job.sql
7
8 # 修改配置文件 application.properties
9 # 设置数据库连接信息
10
11 # 编译并启动
12 mvn clean package
13 cd xxl-job-admin/target
```

 Bash

```
14 java -jar xxl-job-admin-2.4.0.jar
```

访问 <http://localhost:8080/xxl-job-admin>，默认账号密码为 admin/123456。

步骤二：引入依赖

在 Spring Boot 项目的 pom.xml 中添加：

```
1 <dependency>
2   <groupId>com.xuxueli</groupId>
3   <artifactId>xxl-job-core</artifactId>
4   <version>2.4.0</version>
5 </dependency>
```

CAUTION

XXL-JOB 2.4.0 版本兼容 Spring Boot 3.x。如果使用更早版本，可能需要额外配置。

步骤三：配置连接信息

在 application.yml 中配置：

```
1 xxl:
2   job:
3     admin:
4       addresses: http://127.0.0.1:8080/xxl-job-admin # 调度中心地址
5     executor:
6       appname: my-springboot-app # 执行器名称
7       port: 9999 # 执行器端口
8       logpath: /data/applogs/xxl-job/jobhandler # 日志路径
9       logretentiondays: 30 # 日志保留天数
10      accessToken: default_token # 通信令牌（可选）
```

步骤四：创建配置类

```
1 import com.xxl.job.core.executor.impl.XxlJobSpringExecutor;
2 import org.springframework.beans.factory.annotation.Value;
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5
6 @Configuration
7 public class XxlJobConfig {
8
9     @Value("${xxl.job.admin.addresses}")
10    private String adminAddresses;
11
12    @Value("${xxl.job.executor.appname}")
13    private String appName;
14
15    @Value("${xxl.job.executor.port}")
16    private int port;
17
18    @Value("${xxl.job.accessToken}")
```

```
19     private String accessToken;
20
21     @Value("${xxl.job.executor.logpath}")
22     private String logPath;
23
24     @Value("${xxl.job.executor.logretentiondays}")
25     private int logRetentionDays;
26
27     @Bean
28     public XxlJobSpringExecutor xxlJobExecutor() {
29         XxlJobSpringExecutor executor = new XxlJobSpringExecutor();
30         executor.setAdminAddresses(adminAddresses);
31         executor.setAppname(appName);
32         executor.setPort(port);
33         executor.setAccessToken(accessToken);
34         executor.setLogPath(logPath);
35         executor.setLogRetentionDays(logRetentionDays);
36         return executor;
37     }
38 }
```

步骤五：创建任务类

使用 `@XxlJob` 注解定义任务：

```
1  import com.xxl.job.core.handler.annotation.XxlJob;
2  import com.xxl.job.core.context.XxlJobHelper;
3  import org.slf4j.Logger;
4  import org.slf4j.LoggerFactory;
5  import org.springframework.stereotype.Component;
6
7  @Component
8  public class SampleXxlJob {
9
10     private static final Logger logger = LoggerFactory.getLogger(SampleXxlJob.class);
11
12     /**
13      * 简单任务示例
14      */
15     @XxlJob("demoJobHandler")
16     public void demoJobHandler() throws Exception {
17         logger.info("XXL-JOB 任务开始执行");
18
19         // 获取任务参数
20         String param = XxlJobHelper.getJobParam();
21         logger.info("任务参数: {}", param);
22
23         // 执行业务逻辑
24         for (int i = 0; i < 5; i++) {
25             logger.info("执行进度: {}/5", i + 1);
26             Thread.sleep(1000);
27         }
28     }
29 }
```

 Java

```
28
29     // 设置任务结果
30     XxlJobHelper.handleSuccess("任务执行成功");
31     logger.info("XXL-JOB 任务执行完成");
32 }
33
34 /**
35  * 分片任务示例
36  */
37 @XxlJob("shardingJobHandler")
38 public void shardingJobHandler() throws Exception {
39     // 分片参数
40     int shardIndex = XxlJobHelper.getShardIndex();
41     int shardTotal = XxlJobHelper.getShardTotal();
42
43     logger.info("分片任务执行, 当前分片: {}/{}", shardIndex, shardTotal);
44
45     // 根据分片参数处理数据
46     // 例如: 查询 id % shardTotal == shardIndex 的数据
47     List<Data> dataList = dataRepository.findByShard(shardIndex, shardTotal);
48
49     for (Data data : dataList) {
50         processData(data);
51     }
52
53     XxlJobHelper.handleSuccess("分片任务完成");
54 }
55
56 private void processData(Data data) {
57     // 处理单条数据
58 }
59 }
```

步骤六：管理界面配置

1. 登录 XXL-JOB 管理界面
2. 进入「执行器管理」，确认执行器已注册
3. 进入「任务管理」，点击「新增」
4. 配置任务信息：
 - 执行器：选择 my-springboot-app
 - 任务描述：填写任务说明
 - 负责人：填写负责人
 - 调度类型：选择 CRON
 - Cron 表达式：输入 cron 表达式（如 0 0 2 ?）
 - 运行模式：选择 BEAN
 - JobHandler：填写 `@XxlJob` 注解的值（如 demoJobHandler）
 - 路由策略：选择第一个、轮询、故障转移等
5. 保存后可以在任务列表中启动/停止任务

TIP

XXL-JOB 支持在线查看任务执行日志，便于问题排查。建议在任务中使用 logger 记录关键信息。

WARNING

生产环境务必修改默认的 accessToken，并配置防火墙规则限制调度中心的访问权限。

3.3 Quartz 框架集成

Quartz 是一个功能强大的开源作业调度框架，支持复杂的调度需求和持久化。

INFO

Spring Boot 3.2.4 内置支持 Quartz 2.3.2 版本，无需额外指定版本号。

核心概念

概念	说明
Job	执行的任务接口，定义 execute 方法
JobDetail	任务的详细信息，包括名称、组、参数等
Trigger	触发器，定义任务何时执行
Scheduler	调度器，管理 Job 和 Trigger
Calendar	日历，排除特定日期
JobStore	任务存储，支持内存和数据库

Spring Boot 集成 Quartz

步骤一：添加依赖

在 pom.xml 中添加 Quartz starter：

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-quartz</artifactId>
4 </dependency>
```

xml

NOTE

Spring Boot 3.2.4 会自动管理 Quartz 的版本（2.3.2），无需手动指定 version。

步骤二：定义 Job 类

创建实现 `Job` 接口的类：

```
1 import org.quartz.Job;
2 import org.quartz.JobExecutionContext;
3 import org.quartz.JobExecutionException;
4 import org.quartz.JobDataMap;
5 import org.springframework.stereotype.Component;
6 import java.time.LocalDateTime;
```

Java

```
7
8 @Component
9 public class ReportJob implements Job {
10
11     @Override
12     public void execute(JobExecutionContext context) throws JobExecutionException {
13         System.out.println("Quartz 任务执行: " + LocalDateTime.now());
14
15         // 获取任务参数
16         JobDataMap dataMap = context.getMergedJobDataMap();
17         String reportType = dataMap.getString("reportType");
18
19         System.out.println("报表类型: " + reportType);
20
21         // 执行业务逻辑
22         generateReport(reportType);
23     }
24
25     private void generateReport(String reportType) {
26         // 生成报表逻辑
27         System.out.println("生成 " + reportType + " 报表");
28     }
29 }
```

CAUTION

Job 类必须由无参构造函数，且应该是线程安全的（因为可能被多个线程同时执行）。

步骤三：配置 JobDetail 和 Trigger

创建配置类定义任务的详细信息和触发器：

```
1 import org.quartz.*;
2 import org.springframework.context.annotation.Bean;
3 import org.springframework.context.annotation.Configuration;
4
5 @Configuration
6 public class QuartzConfig {
7
8     @Bean
9     public JobDetail reportJobDetail() {
10         return JobBuilder.newJob(ReportJob.class)
11             .withIdentity("reportJob", "reportGroup")
12             .withDescription("日报生成任务")
13             .storeDurably() // 即使没有 Trigger 关联也持久化
14             .build();
15     }
16
17     @Bean
18     public Trigger reportJobTrigger() {
19         // 每天凌晨 2 点执行
20         CronScheduleBuilder scheduleBuilder = CronScheduleBuilder
21             .cronSchedule("0 0 2 * * ?");
```

 Java

```
22
23     return TriggerBuilder.newTrigger()
24         .forJob(reportJobDetail())
25         .withIdentity("reportTrigger", "reportGroup")
26         .withDescription("每天凌晨2点触发")
27         .withSchedule(scheduleBuilder)
28         .build();
29     }
30 }
```

TIP

`withIdentity(name, group)` 中的 `group` 参数可以用于对任务进行分类管理。

Quartz 持久化配置

使用数据库存储任务信息，支持集群模式：

步骤一：配置 `application.yml`

```
1  spring:
2    quartz:
3      job-store-type: jdbc # 使用数据库存储
4      jdbc:
5        initialize-schema: always # 自动创建表结构（生产环境建议改为 never）
6        properties:
7          org:
8            quartz:
9              scheduler:
10               instanceName: clusteredScheduler
11               instanceId: AUTO # 自动生成实例 ID
12             jobStore:
13               class: org.quartz.impl.jdbcjobstore.JobStoreTX
14               driverDelegateClass: org.quartz.impl.jdbcjobstore.StdJDBCDelegate
15               tablePrefix: QRTZ_ # 表前缀
16               isClustered: true # 启用集群模式
17               clusterCheckinInterval: 10000 # 集群检查间隔（毫秒）
18               useProperties: false
19             threadPool:
20               class: org.quartz.simpl.SimpleThreadPool
21               threadCount: 10 # 线程池大小
22               threadPriority: 5
```

INFO

Quartz 持久化需要执行官方提供的 SQL 脚本创建 11 张表，Spring Boot 可以自动初始化（`initialize-schema: always`）。生产环境建议将 SQL 脚本保存到版本控制中，并将此选项设为 `never`。

步骤二：数据源配置

确保项目中配置了数据源（以 MySQL 为例）：

```
1  spring:
2    datasource:
```

```
3 url: jdbc:mysql://localhost:3306/quartz_db?useSSL=false&serverTimezone=UTC
4 username: root
5 password: your_password
6 driver-class-name: com.mysql.cj.jdbc.Driver
```

添加 MySQL 驱动依赖：

```
1 <dependency>
2   <groupId>com.mysql</groupId>
3   <artifactId>mysql-connector-j</artifactId>
4   <scope>runtime</scope>
5 </dependency>
```

步骤三：验证表结构

启动应用后，数据库中会自动创建以下 11 张表：

表名	用途
QRTZ_CALENDARS	存储 Calendar 信息
QRTZ_CRON_TRIGGERS	存储 CronTrigger 信息
QRTZ_FIRED_TRIGGERS	存储已触发的 Trigger
QRTZ_JOB_DETAILS	存储 JobDetail 信息
QRTZ_LOCKS	存储锁信息
QRTZ_PAUSED_TRIGGER_GRPS	存储暂停的 Trigger 组
QRTZ_SCHEDULER_STATE	存储调度器状态
QRTZ_SIMPLE_TRIGGERS	存储 SimpleTrigger 信息
QRTZ_SIMPROP_TRIGGERS	存储 SimpropTrigger 信息
QRTZ_TRIGGERS	存储 Trigger 基本信息
QRTZ_BLOB_TRIGGERS	存储 BlobTrigger 信息

WARNING

集群模式下，所有节点必须使用相同的数据源配置。确保数据库连接稳定，否则可能导致任务重复执行或丢失。

SimpleTrigger 与 CronTrigger

Quartz 提供两种主要的触发器类型：

SimpleTrigger：简单的周期性触发

```
1 @Bean
2 public Trigger simpleTrigger() {
3     return TriggerBuilder.newTrigger()
4         .forJob(simpleJobDetail())
5         .withIdentity("simpleTrigger")
6         .startAt(new Date(System.currentTimeMillis() + 5000)) // 5秒后开始
7         .withSchedule(SimpleScheduleBuilder.simpleSchedule())
```



```
8         .withIntervalInSeconds(10) // 每10秒执行
9         .repeatForever()           // 无限重复
10        .build();
11    }
```

CronTrigger: 基于 Cron 表达式的触发

```
1 @Bean
2 public Trigger cronTrigger() {
3     return TriggerBuilder.newTrigger()
4         .forJob(cronJobDetail())
5         .withIdentity("cronTrigger")
6         .withSchedule(CronScheduleBuilder.cronSchedule("0 0/5 * * * ?"))
7         .build();
8 }
```

特性	SimpleTrigger	CronTrigger
适用场景	固定间隔执行	复杂时间规则
配置难度	简单	需要掌握 Cron 语法
灵活性	较低	高
示例	每 5 分钟执行	工作日 9-17 点每小时执行

动态管理 Quartz 任务

通过 `Scheduler` API 可以在运行时动态管理任务:

```
1 import org.quartz.Scheduler;
2 import org.quartz.SchedulerException;
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Service;
5
6 @Service
7 public class QuartzTaskService {
8
9     @Autowired
10     private Scheduler scheduler;
11
12     /**
13      * 添加任务
14      */
15     public void addJob(String jobName, String jobGroup, String cronExpression)
16         throws SchedulerException {
17         JobDetail jobDetail = JobBuilder.newJob(ReportJob.class)
18             .withIdentity(jobName, jobGroup)
19             .build();
20
21         CronTrigger trigger = TriggerBuilder.newTrigger()
22             .withIdentity(jobName + "_trigger", jobGroup)
23             .withSchedule(CronScheduleBuilder.cronSchedule(cronExpression))
24             .build();
25     }
```

```
26     scheduler.scheduleJob(jobDetail, trigger);
27 }
28
29 /**
30  * 暂停任务
31  */
32 public void pauseJob(String jobName, String jobGroup) throws SchedulerException {
33     scheduler.pauseJob(JobKey.jobKey(jobName, jobGroup));
34 }
35
36 /**
37  * 恢复任务
38  */
39 public void resumeJob(String jobName, String jobGroup) throws SchedulerException {
40     scheduler.resumeJob(JobKey.jobKey(jobName, jobGroup));
41 }
42
43 /**
44  * 删除任务
45  */
46 public void deleteJob(String jobName, String jobGroup) throws SchedulerException {
47     scheduler.deleteJob(JobKey.jobKey(jobName, jobGroup));
48 }
49
50 /**
51  * 立即执行任务
52  */
53 public void triggerJob(String jobName, String jobGroup) throws SchedulerException {
54     scheduler.triggerJob(JobKey.jobKey(jobName, jobGroup));
55 }
56 }
```

TIP

动态管理功能适合需要用户自定义任务时间的场景，如用户设置的提醒、定期报表等。

4 异步编程最佳实践

4.1 异常处理机制

异步编程中的异常处理比同步代码更复杂，因为异常不会直接传播到调用线程。

全局异常处理器

为异步线程池配置全局异常处理器：

```
1 import org.springframework.aop.interceptor.AsyncUncaughtExceptionHandler;
2 import org.springframework.context.annotation.Configuration;
3 import org.springframework.scheduling.annotation.AsyncConfigurer;
4 import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;
5 import java.lang.reflect.Method;
```

 Java

```
6  import java.util.concurrent.Executor;
7
8  @Configuration
9  public class AsyncExceptionConfig implements AsyncConfigurer {
10
11      @Override
12      public Executor getAsyncExecutor() {
13          ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
14          executor.setCorePoolSize(5);
15          executor.setMaxPoolSize(10);
16          executor.setQueueCapacity(100);
17          executor.setThreadNamePrefix("async-");
18
19          // 设置异常处理器
20          executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
21
22          executor.initialize();
23          return executor;
24      }
25
26      @Override
27      public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
28          return new CustomAsyncExceptionHandler();
29      }
30  }
31
32  class CustomAsyncExceptionHandler implements AsyncUncaughtExceptionHandler {
33
34      private static final Logger log =
35          LoggerFactory.getLogger(CustomAsyncExceptionHandler.class);
36
37      @Override
38      public void handleUncaughtException(Throwable ex, Method method, Object... params) {
39          log.error("异步方法 {} 执行异常, 参数: {}", method.getName(), params, ex);
40
41          // 可以发送邮件告警、记录监控指标等
42          // alertService.sendAlert(ex.getMessage());
43      }
44  }
```

CompletableFuture 异常处理

```
1  @Service
2  public class ExceptionHandlingService {
3
4      @Async
5      public CompletableFuture<String> riskyOperation() {
6          return CompletableFuture.supplyAsync(() -> {
7              // 可能抛出异常的操作
8              if (Math.random() > 0.5) {
9                  throw new RuntimeException("业务异常");
10             }
11             return "成功";
12         });
13     }
14 }
```



```
12     }).exceptionally(ex -> {
13         log.error("异步操作失败", ex);
14         return "降级结果";
15     });
16 }
17
18 // 链式异常处理
19 public CompletableFuture<String> chainedWithErrorHandling() {
20     return asyncMethod1()
21         .thenCompose(result1 -> asyncMethod2(result1))
22         .thenApply(result2 -> transform(result2))
23         .exceptionally(ex -> {
24             log.error("链式调用失败", ex);
25             return "默认值";
26         });
27 }
28 }
```

CAUTION

不要在 `exceptionally` 中再次抛出异常，这会导致异常被吞掉。应该返回一个合理的默认值或错误标识。

超时控制

为异步操作设置超时，防止长时间阻塞：

```
1 @GetMapping("/timeout")
2 public String handleWithTimeout() {
3     CompletableFuture<String> future = asyncService.slowOperation();
4
5     try {
6         // 设置 3 秒超时
7         String result = future.get(3, TimeUnit.SECONDS);
8         return result;
9     } catch (TimeoutException e) {
10         future.cancel(true); // 取消任务
11         return "操作超时";
12     } catch (Exception e) {
13         return "执行失败: " + e.getMessage();
14     }
15 }
```

 Java

或者使用 `orTimeout` (Java 9+)：

```
1 CompletableFuture<String> result = asyncService.slowOperation()
2     .orTimeout(3, TimeUnit.SECONDS)
3     .exceptionally(ex -> "超时或失败: " + ex.getMessage());
```

 Java

4.2 上下文传递

异步执行时，`ThreadLocal` 中的上下文信息（如用户信息、请求 ID、事务上下文等）不会自动传递到子线程。

问题演示

```
1 // 主线程设置上下文
2 RequestContext.setUser(currentUser);
3
4 // 异步方法中无法获取
5 @Async
6 public void asyncMethod() {
7     User user = RequestContext.getUser(); // null!
8 }
```

Java

解决方案一：手动传递参数

最简单直接的方式是通过方法参数传递：

```
1 @Async
2 public void asyncMethodWithContext(User user, String requestId) {
3     // 直接使用传入的参数
4     log.info("用户: {}, 请求ID: {}", user.getUsername(), requestId);
5     // 业务逻辑
6 }
```

Java

TIP

这是最推荐的方式，简单可靠，没有额外的复杂性。

解决方案二：使用 TaskDecorator

Spring 5.0 引入了 `TaskDecorator`，可以在任务提交到线程池时复制上下文：

```
1 import org.springframework.core.task.TaskDecorator;
2
3 public class ContextCopyingDecorator implements TaskDecorator {
4
5     @Override
6     public Runnable decorate(Runnable runnable) {
7         // 捕获主线程的上下文
8         RequestAttributes context = RequestContextHolder.currentRequestAttributes();
9         User currentUser = RequestContext.getCurrentUser();
10        String requestId = MDC.get("requestId");
11
12        return () -> {
13            try {
14                // 在子线程中恢复上下文
15                RequestContextHolder.setRequestAttributes(context);
16                RequestContext.setCurrentUser(currentUser);
17                MDC.put("requestId", requestId);
18
19                // 执行原始任务
20                runnable.run();
21            } finally {
22                // 清理上下文，防止内存泄漏
23                RequestContextHolder.resetRequestAttributes();
24                MDC.clear();
25            }
26        };
27    }
28 }
```

Java

```
25     }  
26     };  
27 }  
28 }
```

配置线程池使用装饰器：

```
1  @Bean("taskExecutor")  
2  public Executor taskExecutor() {  
3      ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();  
4      executor.setCorePoolSize(5);  
5      executor.setMaxPoolSize(10);  
6      executor.setQueueCapacity(100);  
7      executor.setThreadNamePrefix("async-");  
8  
9      // 设置任务装饰器  
10     executor.setTaskDecorator(new ContextCopyingDecorator());  
11  
12     executor.initialize();  
13     return executor;  
14 }
```

❶ 解决方案三：使用 InheritableThreadLocal

对于简单的场景，可以使用 `InheritableThreadLocal` 代替 `ThreadLocal`：

```
1  public class RequestContext {  
2      // 使用 InheritableThreadLocal  
3      private static final InheritableThreadLocal<User> currentUser =  
4          new InheritableThreadLocal<>();  
5  
6      public static void setUser(User user) {  
7          currentUser.set(user);  
8      }  
9  
10     public static User getUser() {  
11         return currentUser.get();  
12     }  
13  
14     public static void clear() {  
15         currentUser.remove();  
16     }  
17 }
```

CAUTION

`InheritableThreadLocal` 只在线程创建时复制父线程的值，如果父线程后续修改了值，子线程不会感知。且在线程池场景下，线程复用会导致上下文混乱，不推荐在生产环境使用。

❶ MDC 上下文传递

在日志系统中，MDC（Mapped Diagnostic Context）用于追踪请求链路：

```
1  import org.slf4j.MDC;
```

```
2
3 // 过滤器中设置 MDC
4 @Component
5 public class RequestIdFilter implements Filter {
6
7     @Override
8     public void doFilter(ServletRequest request, ServletResponse response,
9                         FilterChain chain) throws IOException, ServletException {
10         String requestId = UUID.randomUUID().toString();
11         MDC.put("requestId", requestId);
12
13         try {
14             chain.doFilter(request, response);
15         } finally {
16             MDC.clear();
17         }
18     }
19 }
```

配合 `TaskDecorator` 实现 MDC 在异步线程中的传递（见上文示例）。

4.3 性能调优与监控

线程池参数调优

根据任务类型选择合适的线程池配置：

任务类型	CPU 密集型	IO 密集型	混合型
特点	计算密集，少等待	大量 IO 操作，多等待	既有计算又有 IO
核心线程数	CPU 核数 + 1	CPU 核数 × 2	根据实际情况调整
最大线程数	CPU 核数 + 1	CPU 核数 × 4	CPU 核数 × 2~4
队列容量	较小（10-50）	较大（100-1000）	适中（50-200）
示例场景	加密解密、图像处理	数据库查询、HTTP 请求	一般业务逻辑

计算公式参考：

- CPU 密集型： $N(\text{cpu}) + 1$
- IO 密集型： $N(\text{cpu}) / (1 - \text{阻塞系数})$ ，阻塞系数通常在 0.8 0.9 之间

INFO

实际生产中，建议通过压测和监控来确定最优参数，理论公式仅供参考。

监控指标

关键监控指标：

指标	说明	告警阈值
活跃线程数	当前正在执行任务的线程数	接近最大线程数
队列大小	等待执行的任务数量	超过队列容量的 80%

任务完成时间	平均任务执行时长	超过预期时间
拒绝次数	被拒绝执行的任务数	大于 0
线程池活跃度	活跃线程数 / 最大线程数	持续高于 80%

实现监控：

```
1 import io.micrometer.core.instrument.MeterRegistry;
2 import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;
3 import org.springframework.stereotype.Component;
4 import javax.annotation.PostConstruct;
5
6 @Component
7 public class ThreadPoolMonitor {
8
9     @Autowired
10    private ThreadPoolTaskExecutor taskExecutor;
11
12    @Autowired
13    private MeterRegistry meterRegistry;
14
15    @PostConstruct
16    public void init() {
17        // 注册监控指标
18        meterRegistry.gauge("thread.pool.active", taskExecutor,
19            executor -> executor.getActiveCount());
20        meterRegistry.gauge("thread.pool.queue.size", taskExecutor,
21            executor -> executor.getThreadPoolExecutor().getQueue().size());
22        meterRegistry.gauge("thread.pool.completed.tasks", taskExecutor,
23            executor -> (double) executor.getThreadPoolExecutor().getCompletedTaskCount());
24        meterRegistry.gauge("thread.pool.rejected.tasks", taskExecutor,
25            executor -> (double)
26                executor.getThreadPoolExecutor().getRejectedExecutionHandler());
27    }
28 }
```

配合 Prometheus + Grafana 可以实现可视化监控和告警。

常见问题与优化

问题 1：线程池耗尽

现象：任务被拒绝或长时间等待。

解决方案：

- 增加线程池大小
- 优化任务执行时间
- 使用有界队列 + 合适的拒绝策略
- 考虑任务拆分和并行化

问题 2：内存泄漏

现象：应用运行一段时间后 OOM。

原因：

- ThreadLocal 未清理
- 任务持有大对象引用
- 队列积压过多任务

解决方案：

- 确保在 finally 块中清理 ThreadLocal
- 及时释放大对象引用
- 设置合理的队列容量

问题 3：任务堆积

现象：队列持续增长，响应变慢。

解决方案：

- 增加消费者（线程数）
- 优化单个任务性能
- 实现背压机制（拒绝新任务）
- 考虑使用消息队列削峰填谷

WARNING

定期审查线程池的运行状态，通过日志和监控及时发现潜在问题。不要等到生产环境出现故障才进行优化。

调试技巧

技巧	说明
线程命名	为线程池设置有意义的名称前缀，便于排查
日志增强	在异步方法入口和出口记录日志，包含线程名和时间戳
超时检测	为长时间运行的任务设置超时和告警
链路追踪	使用 requestId 贯穿整个调用链
单元测试	编写异步方法的单元测试，验证异常处理和返回值

示例：增强的日志记录

```
1  @Async
2  public CompletableFuture<String> tracedAsyncMethod(String param) {
3      String threadName = Thread.currentThread().getName();
4      String requestId = MDC.get("requestId");
5      long startTime = System.currentTimeMillis();
6
7      log.info("[{}] 异步任务开始, requestId={}, param={}",
8              threadName, requestId, param);
9
10     try {
11         // 业务逻辑
12         String result = doBusinessLogic(param);
13
14         long duration = System.currentTimeMillis() - startTime;
15         log.info("[{}] 异步任务完成, duration={}ms", threadName, duration);
16
17         return CompletableFuture.completedFuture(result);
18     } catch (Exception e) {
```

```
19     long duration = System.currentTimeMillis() - startTime;
20     log.error("[{}] 异步任务失败, duration={}ms", threadName, duration, e);
21
22     CompletableFuture<String> future = new CompletableFuture<>();
23     future.completeExceptionally(e);
24     return future;
25 }
26 }
```

5 本章小结

通过本章的学习，我们掌握了 Spring Boot 中定时任务和异步处理的核心知识：

5.1 定时任务

- 基础使用：@Scheduled 注解支持 fixedRate、fixedDelay 和 Cron 表达式
- 动态任务：通过 SchedulingConfigurer 或 TaskScheduler 实现运行时调整
- 分布式任务：使用 Redis 分布式锁、XXL-JOB 或 Quartz 集群解决多实例问题
- Quartz 集成：功能强大的企业级调度框架，支持持久化和复杂调度

5.2 异步处理

- @Async 注解：简单实现方法异步执行
- 线程池配置：自定义 ThreadPoolTaskExecutor，避免使用默认的 SimpleAsyncTaskExecutor
- CompletableFuture：强大的异步编程工具，支持链式调用和组合
- 上下文传递：通过 TaskDecorator 或参数传递解决 ThreadLocal 失效问题

5.3 最佳实践

- 异常处理：配置全局异常处理器，使用 CompletableFuture.exceptionally()
- 性能调优：根据任务类型（CPU/IO 密集型）合理配置线程池参数
- 监报告警：使用 Micrometer + Prometheus 监控线程池状态
- 日志追踪：通过 MDC 和 requestId 实现完整的链路追踪

TIP

在实际项目中，建议：

- 优先使用 Spring Boot 内置的 @Scheduled 和 @Async
- 对于复杂的调度需求，考虑使用 Quartz 或 XXL-JOB
- 始终配置自定义线程池，不要使用默认配置
- 做好异常处理和监控，便于问题排查

WARNING

异步编程虽然能提升性能，但也增加了系统的复杂性。在使用前务必评估是否真的需要异步，避免过度设计。

本章完

...

Chapter 4

日志系统详解

日志是应用程序的“黑匣子”，记录系统运行状态。Spring Boot 的日志设计遵循门面模式，通过 SLF4J 统一接口，底层可以灵活切换实现框架，实现了日志系统的解耦和可扩展性。

1 Java 日志生态与 SLF4J 门面模式

1.1 Java 日志生态演变

在 SLF4J 出现之前，Java 日志领域存在多个竞争框架：

框架	发布年份	特点	问题
java.util.logging (JUL)	2002	JDK 自带	功能简单，性能一般
Log4j 1.x	2001	功能强大	已停止维护，有安全漏洞
Logback	2006	Log4j 改进版	需要 SLF4J 配合
Log4j 2.x	2014	高性能	API 不兼容 Log4j 1.x

存在的问题：

- 不同库使用不同的日志框架
- 项目中可能出现多个日志实现
- 配置复杂，难以统一管理

1.2 SLF4J 的解决方案

SLF4J (Simple Logging Facade for Java) 是一个日志门面 (Logging Facade)。

核心理念：

1 应用代码 → SLF4J API (接口) → 具体实现 (Logback/Log4j2/JUL)

优势：

1. 解耦：应用代码只依赖 SLF4J API
2. 灵活：可以随时切换底层实现
3. 统一：所有库都通过 SLF4J 输出日志
4. 性能：支持参数化日志，避免不必要的字符串拼接

```
1 // 使用SLF4J API (推荐)
2 import org.slf4j.Logger;
3 import org.slf4j.LoggerFactory;
4
```



```
5 public class UserService {
6     private static final Logger log = LoggerFactory.getLogger(UserService.class);
7
8     public void createUser(String name) {
9         // 参数化日志，只有当日志级别启用时才拼接字符串
10        log.info("Creating user: {}", name);
11    }
12 }
```

TIP

永远使用 SLF4J API，不要直接使用 Logback 或 Log4j 的 API。这样可以在不修改代码的情况下切换日志实现。

2 Spring Boot 默认日志框架

2.1 默认日志框架

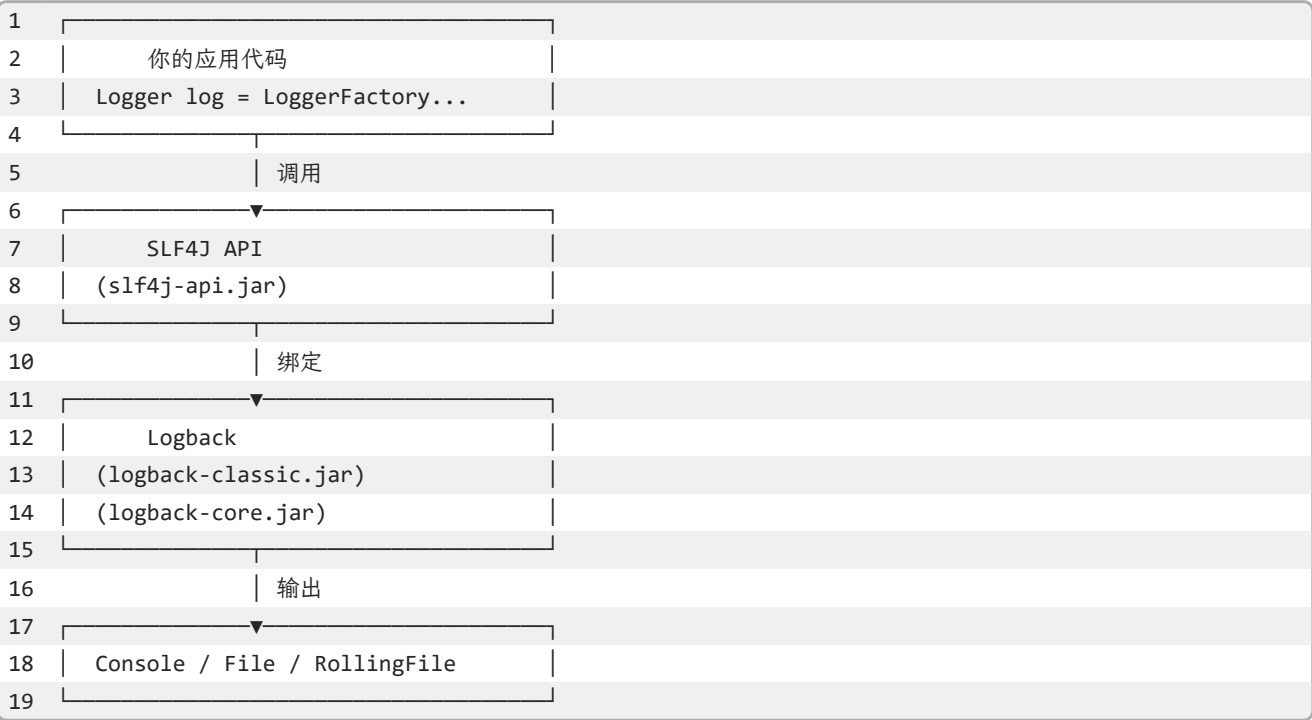
Spring Boot 默认使用 Logback 作为日志实现。

依赖关系：

```
1 <!-- spring-boot-starter -->
2 <dependency>
3     <groupId>org.springframework.boot</groupId>
4     <artifactId>spring-boot-starter</artifactId>
5 </dependency>
6     ↓ 传递依赖
7 <!-- spring-boot-starter-logging -->
8 <dependency>
9     <groupId>org.springframework.boot</groupId>
10    <artifactId>spring-boot-starter-logging</artifactId>
11 </dependency>
12    ↓ 包含
13 <!-- SLF4J API -->
14 <dependency>
15     <groupId>org.slf4j</groupId>
16     <artifactId>slf4j-api</artifactId>
17 </dependency>
18 <!-- Logback Classic (实现) -->
19 <dependency>
20     <groupId>ch.qos.logback</groupId>
21     <artifactId>logback-classic</artifactId>
22 </dependency>
23 <!-- Logback Core -->
24 <dependency>
25     <groupId>ch.qos.logback</groupId>
26     <artifactId>logback-core</artifactId>
27 </dependency>
28 <!-- JUL到SLF4J的桥接 -->
29 <dependency>
```

```
30 <groupId>org.slf4j</groupId>
31 <artifactId>jul-to-slf4j</artifactId>
32 </dependency>
```

架构图：



2.2 日志桥接（Bridge）

Spring Boot 自动配置了日志桥接，将其他日志框架的输出重定向到 SLF4J：

桥接模块	作用	重定向
jcl-over-slf4j	Jakarta Commons Logging	→ SLF4J
jul-to-slf4j	java.util.logging	→ SLF4J
log4j-over-slf4j	Log4j 1.x	→ SLF4J

效果：即使第三方库使用 JUL 或 Log4j，日志也会统一通过 SLF4J 输出。

CAUTION

不要同时引入 `log4j-over-slf4j` 和 `slf4j-log4j12`，会导致循环依赖和 StackOverflowError。

3 Logback 配置详解

3.1 配置文件位置

Spring Boot 按以下顺序查找 Logback 配置：

- 1. `logback-spring.xml`（推荐，支持 Spring 特性）
- 2. `logback.xml`

3. application.properties/yml 中的配置

推荐：使用 logback-spring.xml，因为它支持 Spring 的 Profile 和属性占位符。

3.2 基本配置结构

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <configuration>
3
4      <!-- 属性定义 -->
5      <property name="LOG_PATTERN"
6              value="%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"/>
7      <property name="LOG_PATH" value="logs"/>
8
9      <!-- 控制台输出 -->
10     <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
11         <encoder>
12             <pattern>${LOG_PATTERN}</pattern>
13             <charset>UTF-8</charset>
14         </encoder>
15     </appender>
16
17     <!-- 文件输出 -->
18     <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
19         <file>${LOG_PATH}/application.log</file>
20         <encoder>
21             <pattern>${LOG_PATTERN}</pattern>
22         </encoder>
23         <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
24             <fileNamePattern>${LOG_PATH}/application.%d{yyyy-MM-dd}.log</fileNamePattern>
25             <maxHistory>30</maxHistory>
26             <totalSizeCap>1GB</totalSizeCap>
27         </rollingPolicy>
28     </appender>
29
30     <!-- 根日志器 -->
31     <root level="INFO">
32         <appender-ref ref="CONSOLE"/>
33         <appender-ref ref="FILE"/>
34     </root>
35
36     <!-- 特定包的日志级别 -->
37     <logger name="com.example.myapp" level="DEBUG"/>
38     <logger name="org.springframework.web" level="INFO"/>
39     <logger name="org.hibernate.SQL" level="DEBUG"/>
40
41 </configuration>
```

3.3 日志级别

级别	值	用途	生产环境
----	---	----	------

TRACE	0	最详细追踪	禁用
DEBUG	10	调试信息	禁用
INFO	20	重要信息	✓ 推荐
WARN	30	警告信息	✓ 推荐
ERROR	40	错误信息	✓ 推荐
OFF	Integer.MAX_VALUE	关闭日志	特殊场景

继承规则：

- 子 Logger 继承父 Logger 的级别
- 如果未显式设置，使用 root 的级别
- 更具体的配置覆盖通用配置

```
1 <!-- root设置为INFO -->
2 <root level="INFO">
3   <appender-ref ref="CONSOLE"/>
4 </root>
5
6 <!-- com.example包下的类使用DEBUG级别 -->
7 <logger name="com.example" level="DEBUG"/>
8
9 <!-- com.example.service包下的类使用TRACE级别 -->
10 <logger name="com.example.service" level="TRACE"/>
```

3.4 日志格式模式

常用模式字符：

模式	含义	示例
`%d{pattern}`	日期时间	`2024-01-15 10:30:45.123`
`%thread`	线程名	`main`, `http-nio-8080-exec-1`
`%-5level`	日志级别	`INFO`, `ERROR`
`%logger{length}`	Logger 名称	`c.e.m.UserService`
`%msg`	日志消息	`Creating user: Alice`
`%n`	换行符	
`%exception`	异常堆栈	完整堆栈信息

彩色输出（仅控制台）：

```
1 <pattern>%clr(%d{yyyy-MM-dd HH:mm:ss.SSS}){faint}
2   %clr(%5p) %clr(${PID:- }){magenta}
3   %clr(---){faint} %clr([%15.15t]){faint}
4   %clr(%-40.40logger{39}){cyan} %clr(:){faint} %m%n</pattern>
```

TIP

Spring Boot 默认的日志格式已经包含了彩色输出，开发时更易读。

3.5 application.properties 配置

对于简单场景，可以直接在配置文件中设置：

```
1  # 日志级别
2  logging.level.root=INFO
3  logging.level.com.example.myapp=DEBUG
4  logging.level.org.springframework.web=WARN
5
6  # 日志文件
7  logging.file.name=logs/application.log
8  logging.file.max-size=10MB
9  logging.file.max-history=30
10
11 # 日志格式
12 logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n
13 logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
14
15 # 日志组 (Spring Boot 2.1+)
16 logging.group.tomcat=org.apache.catalina,org.apache.coyote,org.apache.tomcat
17 logging.level.tomcat=DEBUG
```

NOTE

对于复杂的日志配置（如多 appender、滚动策略），建议使用 `logback-spring.xml`。

4 异步日志与性能优化

4.1 异步日志

异步日志可以提升性能，特别是高并发场景：

```
1  <appender name="ASYNC_FILE" class="ch.qos.logback.classic.AsyncAppender">
2      <!-- 队列大小 -->
3      <queueSize>512</queueSize>
4      <!-- 丢弃阈值（队列剩余容量低于此值时丢弃TRACE/DEBUG/INFO） -->
5      <discardingThreshold>0</discardingThreshold>
6      <!-- 是否包含调用者信息 -->
7      <includeCallerData>false</includeCallerData>
8
9      <appender-ref ref="FILE"/>
10 </appender>
11
12 <root level="INFO">
13     <appender-ref ref="ASYNC_FILE"/>
14 </root>
```

性能提升：

- 同步日志：约100K msg/s
- 异步日志：约1M msg/s

CAUTION

异步日志在应用关闭时可能丢失少量日志。设置 `queueSize` 时要权衡内存占用。

4.2 条件配置 (Profile)

`logback-spring.xml` 支持 Spring Profile:

```
1 <configuration>
2
3 <!-- 开发环境 -->
4 <springProfile name="dev">
5   <root level="DEBUG">
6     <appender-ref ref="CONSOLE"/>
7   </root>
8   <logger name="com.example" level="TRACE"/>
9 </springProfile>
10
11 <!-- 生产环境 -->
12 <springProfile name="prod">
13   <root level="WARN">
14     <appender-ref ref="ASYNC_FILE"/>
15   </root>
16   <logger name="com.example" level="INFO"/>
17 </springProfile>
18
19 <!-- 测试环境 -->
20 <springProfile name="test">
21   <root level="INFO">
22     <appender-ref ref="CONSOLE"/>
23   </root>
24 </springProfile>
25
26 </configuration>
```

5 结构化日志与 JSON 格式输出

5.1 为什么需要结构化日志

传统日志的问题:

```
1 2024-01-15 10:30:45.123 [http-nio-8080-exec-1] INFO c.e.m.UserService - Creating user: Alice,
  age=25, email=alice@example.com
```

- 难以解析和查询
- 无法高效聚合分析
- 不支持多维度的过滤

结构化日志:

```
1 {
2   "timestamp": "2024-01-15T10:30:45.123Z",
```

```
3   "level": "INFO",
4   "thread": "http-nio-8080-exec-1",
5   "logger": "com.example.myapp.UserService",
6   "message": "Creating user",
7   "user_name": "Alice",
8   "user_age": 25,
9   "user_email": "alice@example.com",
10  "trace_id": "abc123",
11  "span_id": "def456"
12 }
```

优势:

- 易于机器解析
- 支持复杂查询
- 便于日志聚合（ELK、Loki）
- 支持链路追踪

TIP

微服务架构中，结构化日志是标配，强烈推荐使用。

5.2 JSON 格式化日志

使用 logstash-logback-encoder

```
1 <dependency>
2   <groupId>net.logstash.logback</groupId>
3   <artifactId>logstash-logback-encoder</artifactId>
4   <version>7.4</version>
5 </dependency>
```

配置:

```
1 <appender name="JSON_CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
2   <encoder class="net.logstash.logback.encoder.LogstashEncoder">
3     <!-- 自定义字段 -->
4     <customFields>{"app": "myapp", "env": "${APP_ENV:-dev}"}</customFields>
5
6     <!-- 包含MDC -->
7     <includeMdcKeyName>trace_id</includeMdcKeyName>
8     <includeMdcKeyName>user_id</includeMdcKeyName>
9
10    <!-- 时间戳格式 -->
11    <timestampPattern>yyyy-MM-dd'T'HH:mm:ss.SSS'Z'</timestampPattern>
12    <timeZone>UTC</timeZone>
13  </encoder>
14 </appender>
15
16 <root level="INFO">
17   <appender-ref ref="JSON_CONSOLE"/>
18 </root>
```

输出:

```
1 {
2   "@timestamp": "2024-01-15T10:30:45.123Z",
3   "@version": "1",
4   "message": "Creating order",
5   "logger_name": "com.example.myapp.OrderService",
6   "thread_name": "http-nio-8080-exec-1",
7   "level": "INFO",
8   "level_value": 20000,
9   "app": "myapp",
10  "env": "dev",
11  "trace_id": "abc123def456",
12  "user_id": "user123"
13 }
```

6 MDC 与链路追踪上下文

6.1 MDC (Mapped Diagnostic Context)

MDC 是 SLF4J 提供的线程级别的上下文存储，用于在日志中添加额外信息。

基本原理

```
1 请求进入 → 生成trace_id → 放入MDC → 所有日志自动包含trace_id → 请求结束清除MDC
```

基本用法

```
1  import org.slf4j.MDC;
2
3  @Service
4  public class OrderService {
5      private static final Logger log = LoggerFactory.getLogger(OrderService.class);
6
7      public void createOrder(String orderId) {
8          // 将订单ID放入MDC
9          MDC.put("order_id", orderId);
10         MDC.put("user_id", "user123");
11
12         try {
13             log.info("Creating order");
14             // ... 业务逻辑
15             log.info("Order created successfully");
16         } finally {
17             // 清理MDC (重要!)
18             MDC.clear();
19         }
20     }
21 }
```

日志输出：

```
1 2024-01-15 10:30:45.123 [http-nio-8080-exec-1] INFO c.e.m.OrderService - Creating order
  {order_id=ORD001, user_id=user123}
2 2024-01-15 10:30:45.456 [http-nio-8080-exec-1] INFO c.e.m.OrderService - Order created
  successfully {order_id=ORD001, user_id=user123}
```

CAUTION

必须在 finally 块中清理 MDC，否则在线程池环境下会导致内存泄漏和数据污染。

在 Filter 中设置 Trace ID

```
1 import org.slf4j.MDC;
2 import org.springframework.stereotype.Component;
3 import javax.servlet.*;
4 import java.io.IOException;
5 import java.util.UUID;
6
7 @Component
8 public class TraceIdFilter implements Filter {
9
10     private static final String TRACE_ID_KEY = "trace_id";
11
12     @Override
13     public void doFilter(ServletRequest request, ServletResponse response,
14                         FilterChain chain) throws IOException, ServletException {
15         try {
16             // 从请求头获取或生成新的trace_id
17             String traceId = ((HttpServletRequest) request).getHeader("X-Trace-Id");
18             if (traceId == null || traceId.isEmpty()) {
19                 traceId = UUID.randomUUID().toString().replace("-", "");
20             }
21
22             // 放入MDC
23             MDC.put(TRACE_ID_KEY, traceId);
24
25             // 继续处理
26             chain.doFilter(request, response);
27         } finally {
28             // 清理MDC
29             MDC.clear();
30         }
31     }
32 }
```

配置 Logback pattern:

```
1 <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %X{trace_id} -
  %msg%n</pattern>
```

效果:

```
1 2024-01-15 10:30:45.123 [http-nio-8080-exec-1] INFO c.e.m.OrderService - abc123def456 - Creating
  order
2 2024-01-15 10:30:45.456 [http-nio-8080-exec-1] INFO c.e.m.PaymentService - abc123def456 -
  Processing payment
```

TIP

通过 trace_id 可以追踪一个请求在所有服务中的完整调用链路。

7 敏感信息脱敏与安全日志

7.1 为什么需要脱敏

生产环境必须对敏感信息（手机号、身份证、银行卡等）进行脱敏，符合 GDPR 等法规要求。

7.2 实现方式

```
1 import ch.qos.logback.classic.pattern.MessageConverter;
2 import ch.qos.logback.classic.spi.ILoggingEvent;
3
4 public class SensitiveDataConverter extends MessageConverter {
5
6     @Override
7     public String convert(ILoggingEvent event) {
8         String message = event.getFormattedMessage();
9
10        // 脱敏手机号
11        message = message.replaceAll("(\\d{3})\\d{4}(\\d{4})", "$1****$2");
12
13        // 脱敏邮箱
14        message = message.replaceAll("(\\w{3})\\w*@", "$1***@");
15
16        // 脱敏身份证
17        message = message.replaceAll("(\\d{6})\\d{8}(\\d{4})", "$1*****$2");
18
19        return message;
20    }
21 }
```

```
1 <conversionRule conversionWord="msg"
2     converterClass="com.example.SensitiveDataConverter" />
```

NOTE

生产环境必须对敏感信息（手机号、身份证、银行卡等）进行脱敏，符合 GDPR 等法规要求。

8 多环境日志配置

8.1 基于 Profile 的配置

logback-spring.xml 支持 Spring Profile，可以为不同环境配置不同的日志策略：

```
1 <configuration>
2
```

```
3      <!-- 开发环境 -->
4      <springProfile name="dev">
5          <root level="DEBUG">
6              <appender-ref ref="CONSOLE"/>
7          </root>
8          <logger name="com.example" level="TRACE"/>
9      </springProfile>
10
11     <!-- 测试环境 -->
12     <springProfile name="test">
13         <root level="INFO">
14             <appender-ref ref="CONSOLE"/>
15             <appender-ref ref="FILE"/>
16         </root>
17     </springProfile>
18
19     <!-- 生产环境 -->
20     <springProfile name="prod">
21         <root level="WARN">
22             <appender-ref ref="ASYNC_FILE"/>
23         </root>
24         <logger name="com.example" level="INFO"/>
25     </springProfile>
26
27 </configuration>
```

8.2 动态调整日志级别

Spring Boot Actuator 提供了动态调整日志级别的端点：

```
1 # 查看当前日志级别
2 curl http://localhost:8080/actuator/loggers
3
4 # 修改某个包的日志级别
5 curl -X POST http://localhost:8080/actuator/loggers/com.example \
6     -H "Content-Type: application/json" \
7     -d '{"configuredLevel": "DEBUG"}'
```

TIP

生产环境可以通过 Actuator 动态调整日志级别，无需重启应用。

9 自定义 Appender 与日志扩展

9.1 自定义 Appender

可以将日志发送到自定义目标（如 Kafka、Elasticsearch 等）：

```
1 import ch.qos.logback.core.AppenderBase;
2
```

```
3 public class KafkaAppender extends AppenderBase<ILoggingEvent> {
4     private KafkaProducer<String, String> producer;
5     private String topic;
6
7     @Override
8     public void start() {
9         // 初始化Kafka Producer
10        producer = new KafkaProducer<>(...);
11        super.start();
12    }
13
14    @Override
15    protected void append(ILoggingEvent event) {
16        String message = String.format("[%s] %s - %s",
17            event.getLevel(),
18            event.getLoggerName(),
19            event.getFormattedMessage()
20        );
21        producer.send(new ProducerRecord<>(topic, message));
22    }
23
24    @Override
25    public void stop() {
26        producer.close();
27        super.stop();
28    }
29 }
```

```
1 <appender name="KAFKA" class="com.example.KafkaAppender">
2     <topic>application-logs</topic>
3 </appender>
4
5 <root level="INFO">
6     <appender-ref ref="KAFKA"/>
7 </root>
```

xml

9.2 最佳实践总结

1. 统一的日志工具类

```
1 import org.slf4j.Logger;
2 import org.slf4j.LoggerFactory;
3 import org.slf4j.MDC;
4
5 public class LogUtils {
6
7     public static void putTraceId(String traceId) {
8         MDC.put("trace_id", traceId);
9     }
10
11     public static void putUserId(String userId) {
12         MDC.put("user_id", userId);
```

 Java


```
13     }
14
15     public static void putRequestId(String requestId) {
16         MDC.put("request_id", requestId);
17     }
18
19     public static void clear() {
20         MDC.clear();
21     }
22
23     // 便捷方法：记录业务操作
24     public static void logBusinessOperation(Logger log, String operation,
25                                           String status, Object... params) {
26         log.info("operation={}, status={}, params={}", operation, status, params);
27     }
28 }
```

2. AOP 自动添加上下文

```
1  import org.aspectj.lang.ProceedingJoinPoint;
2  import org.aspectj.lang.annotation.Around;
3  import org.aspectj.lang.annotation.Aspect;
4  import org.slf4j.MDC;
5  import org.springframework.stereotype.Component;
6  import java.util.UUID;
7
8  @Aspect
9  @Component
10 public class LoggingAspect {
11
12     @Around("@annotation(org.springframework.web.bind.annotation.RestController)")
13     public Object addLoggingContext(ProceedingJoinPoint joinPoint) throws Throwable {
14         String traceId = UUID.randomUUID().toString().replace("-", "");
15         MDC.put("trace_id", traceId);
16
17         try {
18             return joinPoint.proceed();
19         } finally {
20             MDC.clear();
21         }
22     }
23 }
```

本章详细介绍了 Spring Boot 的日志系统，从 SLF4J 门面模式到 Logback 配置，从异步优化到结构化日志，为构建可观测性系统打下基础。下一章将深入探讨监控与可观测性的其他两大支柱：指标和链路追踪。

Chapter 5

监控与可观测性

可观测性（Observability）是理解系统内部状态的能力，由三大支柱构成：日志（Logs）、指标（Metrics）、链路追踪（Traces）。本章聚焦于指标和追踪，日志已在第 4 章详细讲解。

NOTE

可观测性的目标是：当系统出现问题时，能够通过外部输出快速定位根本原因，而无需修改代码或重新部署。

1 可观测性三大支柱

1.1 Logs（日志）

特点：

- 离散事件记录
- 包含时间戳、级别、消息
- 适合排查具体问题

工具：SLF4J + Logback、ELK、Loki

TIP

日志的详细配置和使用见第 4 章《日志系统详解》。

1.2 Metrics（指标）

特点：

- 数值型数据，可聚合
- 随时间变化的趋势
- 适合监控和告警

类型：

类型	特点	示例
Counter（计数器）	只增不减	请求总数、错误数
Gauge（仪表盘）	可增可减	当前线程数、内存使用
Timer（计时器）	耗时统计	接口响应时间
Histogram（直方图）	分布统计	响应时间分布

工具：Micrometer、Prometheus、Grafana

1.3 Traces（链路追踪）

特点：

- 请求的完整调用链
- 跨服务的分布式追踪
- 适合性能分析和瓶颈定位

核心概念：

概念	说明	示例
Trace	完整的请求链路	一个用户请求的全过程
Span	链路中的一个节点	某个服务的调用
Context	传递的追踪上下文	trace_id、span_id

工具：OpenTelemetry、Zipkin、Jaeger

1.4 三大支柱的关系



协同工作：

1. Metrics 告诉你什么出了问题
2. Traces 告诉你哪里出了问题
3. Logs 告诉你为什么出问题

2 Actuator 端点详解

2.1 什么是 Actuator

Spring Boot Actuator 提供了生产级别的应用监控和管理功能。

核心功能：

- 健康检查（Health）

- 指标收集 (Metrics)
- 环境信息 (Environment)
- 线程转储 (Thread Dump)
- HTTP 追踪 (HTTP Traces)

2.2 启用 Actuator

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-actuator</artifactId>
4 </dependency>
```

```
1 # 暴露所有端点 (生产环境慎用)
2 management.endpoints.web.exposure.include=*
3
4 # 或者只暴露需要的端点
5 management.endpoints.web.exposure.include=health,info,metrics,prometheus
```

2.3 常用端点

端点	路径	作用	生产环境
health	/actuator/health	应用健康状态	✓ 推荐
info	/actuator/info	应用信息	✓ 推荐
metrics	/actuator/metrics	性能指标	✓ 推荐
prometheus	/actuator/prometheus	Prometheus 格式指标	✓ 推荐
env	/actuator/env	环境变量	X 敏感
threaddump	/actuator/threaddump	线程转储	按需
heapdump	/actuator/heapdump	堆转储	按需
loggers	/actuator/loggers	日志配置	按需

2.4 health 端点

基本响应:

```
1 {
2   "status": "UP",
3   "components": {
4     "db": {
5       "status": "UP",
6       "details": {
7         "database": "MySQL",
8         "validationQuery": "isValid()"
9       }
10    },
11    "diskSpace": {
12      "status": "UP",
```

```
13     "details": {
14         "total": 500000000000,
15         "free": 300000000000,
16         "threshold": 10485760
17     }
18 },
19 "redis": {
20     "status": "UP",
21     "details": {
22         "version": "6.2.6"
23     }
24 }
25 }
26 }
```

详细模式：

```
1 # 显示详细信息
2 management.endpoint.health.show-details=always
3 # 或仅在授权时显示
4 management.endpoint.health.show-details=when-authorized
```

properties

2.5 info 端点

自定义应用信息：

```
1 # application.properties
2 info.app.name=My Application
3 info.app.version=1.0.0
4 info.app.description=A Spring Boot application
5
6 # Git信息（需要git-commit-id-plugin）
7 info.git.branch=${git.branch}
8 info.git.commit.id=${git.commit.id}
9 info.git.commit.time=${git.commit.time}
```

properties

响应：

```
1 {
2     "app": {
3         "name": "My Application",
4         "version": "1.0.0",
5         "description": "A Spring Boot application"
6     },
7     "git": {
8         "branch": "main",
9         "commit": {
10             "id": "abc123",
11             "time": "2024-01-15T10:30:00Z"
12         }
13     }
14 }
```

{ }JSON

2.6 metrics 端点

查看所有可用指标:

```
1 curl http://localhost:8080/actuator/metrics  Bash
```

查看具体指标:

```
1 # JVM内存使用  Bash
2 curl http://localhost:8080/actuator/metrics/jvm.memory.used
3
4 # HTTP请求数
5 curl http://localhost:8080/actuator/metrics/http.server.requests
6
7 # 带标签查询
8 curl "http://localhost:8080/actuator/metrics/http.server.requests?tag=status:200"
```

常用 JVM 指标:

指标	说明
<code>`jvm.memory.used`</code>	JVM 内存使用
<code>`jvm.gc.pause`</code>	GC 暂停时间
<code>`jvm.threads.live`</code>	活跃线程数
<code>`process.cpu.usage`</code>	CPU 使用率
<code>`system.load.average.1m`</code>	系统负载

3 自定义健康检查

3.1 HealthIndicator 接口

实现自定义健康检查逻辑:

```
1 import org.springframework.boot.actuate.health.Health;  Java
2 import org.springframework.boot.actuate.health.HealthIndicator;
3 import org.springframework.stereotype.Component;
4
5 @Component
6 public class DatabaseHealthIndicator implements HealthIndicator {
7
8     @Override
9     public Health health() {
10         try {
11             // 执行健康检查逻辑
12             boolean databaseIsUp = checkDatabaseConnection();
13
14             if (databaseIsUp) {
15                 return Health.up()
16                     .withDetail("database", "MySQL");
17             }
18         } catch (Exception e) {
19             return Health.down().withDetail(e.getMessage());
20         }
21     }
22 }
```

```
17         .withDetail("status", "Connected")
18         .build();
19     } else {
20         return Health.down()
21             .withDetail("database", "MySQL")
22             .withDetail("error", "Connection failed")
23             .build();
24     }
25     } catch (Exception e) {
26         return Health.down(e)
27             .withDetail("database", "MySQL")
28             .build();
29     }
30 }
31
32 private boolean checkDatabaseConnection() {
33     // 检查数据库连接
34     return true;
35 }
36 }
```

3.2 AbstractHealthIndicator

更简洁的实现方式:

```
1  import org.springframework.boot.actuate.health.AbstractHealthIndicator;
2  import org.springframework.boot.actuate.health.Health;
3  import org.springframework.stereotype.Component;
4
5  @Component
6  public class RedisHealthIndicator extends AbstractHealthIndicator {
7
8      @Override
9      protected void doHealthCheck(Health.Builder builder) throws Exception {
10         try {
11             // 检查Redis连接
12             redisTemplate.opsForValue().get("health_check");
13             builder.up()
14                 .withDetail("redis", "Connected")
15                 .withDetail("version", redisServerVersion());
16         } catch (Exception e) {
17             builder.down()
18                 .withDetail("redis", "Disconnected")
19                 .withDetail("error", e.getMessage());
20         }
21     }
22
23     private String redisServerVersion() {
24         return "6.2.6";
25     }
26 }
```

3.3 复合健康检查

组合多个检查结果：

```
1  @Component Java
2  public class CompositeHealthIndicator implements HealthIndicator {
3
4      @Autowired
5      private DataSource dataSource;
6
7      @Autowired
8      private RedisTemplate<String, String> redisTemplate;
9
10     @Override
11     public Health health() {
12         Health.Builder builder = Health.up();
13
14         // 检查数据库
15         builder.withDetail("database", checkDatabase());
16
17         // 检查Redis
18         builder.withDetail("redis", checkRedis());
19
20         // 检查外部API
21         builder.withDetail("external_api", checkExternalApi());
22
23         // 如果有任何一个DOWN，整体状态为DOWN
24         if (hasAnyDown(builder.build())) {
25             return Health.down(builder.build().getDetails()).build();
26         }
27
28         return builder.build();
29     }
30
31     private String checkDatabase() {
32         try {
33             dataSource.getConnection().close();
34             return "UP";
35         } catch (SQLException e) {
36             return "DOWN: " + e.getMessage();
37         }
38     }
39
40     private String checkRedis() {
41         try {
42             redisTemplate.opsForValue().get("test");
43             return "UP";
44         } catch (Exception e) {
45             return "DOWN: " + e.getMessage();
46         }
47     }
48
49     private String checkExternalApi() {
```



```
50      // 检查外部API
51      return "UP";
52  }
53
54  private boolean hasAnyDown(Health health) {
55      return health.getDetails().values().stream()
56          .anyMatch(detail -> detail.toString().contains("DOWN"));
57  }
58  }
```

TIP

自定义健康检查应该轻量快速，避免阻塞 health 端点的响应。

4 自定义指标收集

4.1 MeterRegistry

MeterRegistry 是 Micrometer 的核心接口，用于注册和收集指标。

自动注入：

```
1  import io.micrometer.core.instrument.MeterRegistry;
2  import org.springframework.stereotype.Service;
3
4  @Service
5  public class OrderService {
6
7      private final MeterRegistry meterRegistry;
8
9      public OrderService(MeterRegistry meterRegistry) {
10         this.meterRegistry = meterRegistry;
11     }
12 }
```

 Java

4.2 Counter（计数器）

用于统计只增不减的数值：

```
1  @Service
2  public class OrderService {
3
4      private final Counter orderCounter;
5      private final Counter errorCounter;
6
7      public OrderService(MeterRegistry meterRegistry) {
8          // 创建计数器
9          this.orderCounter = Counter.builder("orders.total")
10              .description("Total number of orders")
11              .tag("type", "all")
12              .register(meterRegistry);
```

 Java

```
13
14     this.errorCounter = Counter.builder("orders.errors")
15         .description("Number of failed orders")
16         .register(meterRegistry);
17     }
18
19     public void createOrder(Order order) {
20         try {
21             // 业务逻辑
22             orderCounter.increment();
23         } catch (Exception e) {
24             errorCounter.increment();
25             throw e;
26         }
27     }
28 }
```

快捷方式：

```
1 // 简单计数
2 meterRegistry.counter("orders.total").increment();
3
4 // 带标签
5 meterRegistry.counter("orders.total", "status", "success").increment();
6
7 // 增加指定数量
8 meterRegistry.counter("bytes.received").increment(1024);
```

 Java

4.3 Gauge（仪表盘）

用于监控可增可减的数值：

```
1 @Service
2 public class CacheService {
3
4     private final AtomicInteger cacheSize = new AtomicInteger(0);
5
6     public CacheService(MeterRegistry meterRegistry) {
7         // 注册Gauge，绑定到AtomicInteger
8         Gauge.builder("cache.size", cacheSize, AtomicInteger::get)
9             .description("Current cache size")
10            .register(meterRegistry);
11    }
12
13    public void addToCache(String key, Object value) {
14        cache.put(key, value);
15        cacheSize.incrementAndGet();
16    }
17
18    public void removeFromCache(String key) {
19        cache.remove(key);
20        cacheSize.decrementAndGet();
21    }
22 }
```

 Java

```
22 }
```

监控集合大小：

```
1 List<String> activeConnections = new ArrayList<>();
2
3 Gauge.builder("connections.active", activeConnections, List::size)
4   .register(meterRegistry);
```

 Java

4.4 Timer (计时器)

用于统计耗时：

```
1 @Service
2 public class PaymentService {
3
4     private final Timer paymentTimer;
5
6     public PaymentService(MeterRegistry meterRegistry) {
7         this.paymentTimer = Timer.builder("payment.processing.time")
8             .description("Time taken to process payments")
9             .publishPercentiles(0.5, 0.95, 0.99) // P50, P95, P99
10            .register(meterRegistry);
11    }
12
13    public void processPayment(Payment payment) {
14        // 方式1: record包裹
15        paymentTimer.record(() -> {
16            // 支付处理逻辑
17            doProcessPayment(payment);
18        });
19    }
20
21    public void processPaymentAsync(Payment payment) {
22        // 方式2: 手动计时
23        long start = System.nanoTime();
24        try {
25            doProcessPayment(payment);
26        } finally {
27            long end = System.nanoTime();
28            paymentTimer.record(end - start, TimeUnit.NANOSECONDS);
29        }
30    }
31 }
```

 Java

AOP 方式 (`@Timed` 注解)：

```
1 import io.micrometer.core.annotation.Timed;
2
3 @Service
4 public class OrderService {
5
6     @Timed(value = "order.creation.time", description = "Time to create order")
```

 Java

```
7 public Order createOrder(OrderRequest request) {
8     // 自动记录方法执行时间
9     return orderRepository.save(request.toOrder());
10 }
11 }
```

```
1 // 启用@Timed注解
2 @EnableTimed
3 @SpringBootApplication
4 public class Application {
5     public static void main(String[] args) {
6         SpringApplication.run(Application.class, args);
7     }
8 }
```

 Java

4.5 DistributionSummary (分布摘要)

用于统计非时间类的分布数据：

```
1 @Service
2 public class FileUploadService {
3
4     private final DistributionSummary fileSizeSummary;
5
6     public FileUploadService(MeterRegistry meterRegistry) {
7         this.fileSizeSummary = DistributionSummary.builder("file.upload.size")
8             .description("Distribution of uploaded file sizes")
9             .baseUnit("bytes")
10            .publishPercentiles(0.5, 0.95, 0.99)
11            .register(meterRegistry);
12     }
13
14     public void uploadFile(MultipartFile file) {
15         long size = file.getSize();
16         fileSizeSummary.record(size);
17
18         // 保存文件
19         saveFile(file);
20     }
21 }
```

 Java

5 Micrometer 核心概念与抽象层

5.1 Micrometer 是什么

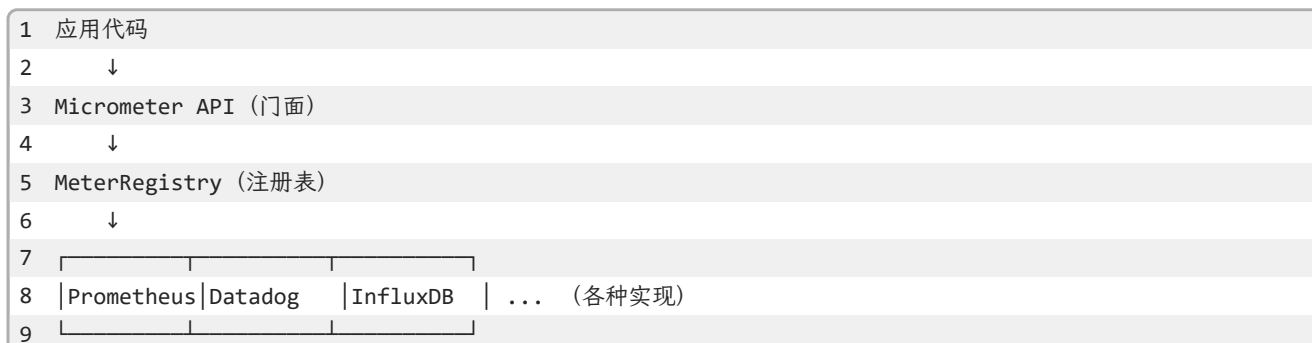
Micrometer 是 JVM 应用的**指标门面** (Metrics Facade)，类似于 SLF4J 之于日志。

优势：

- 统一 API：一套代码支持多种监控系统
- 自动集成：Spring Boot 自动配置

- 丰富类型：Counter、Gauge、Timer 等
- 多维标签：支持 Tags 进行多维度分析

5.2 架构设计



5.3 核心概念

| Meter

Meter 是所有指标类型的父接口：

类型	用途	示例
Counter	递增计数	请求数、错误数
Gauge	当前值	线程数、队列大小
Timer	耗时+计数	接口响应时间
DistributionSummary	分布统计	文件大小、订单金额
LongTaskTimer	长任务计时	批处理任务

| Tags (标签)

Tags 允许对指标进行多维度分析：

```
1 // 不带标签
2 Counter counter = meterRegistry.counter("http.requests");
3
4 // 带标签
5 Counter successCounter = meterRegistry.counter("http.requests",
6     "status", "200",
7     "method", "GET",
8     "uri", "/api/users"
9 );
10
11 Counter errorCounter = meterRegistry.counter("http.requests",
12     "status", "500",
13     "method", "POST",
14     "uri", "/api/orders"
15 );
```

查询时按标签过滤：

```
1 # Prometheus查询
2 curl "http://localhost:8080/actuator/metrics/http.requests?tag=status:200"
```

 Bash

Common Tags（公共标签）

为所有指标添加公共标签：

```
1 # application.properties
2 management.metrics.tags.application=${spring.application.name}
3 management.metrics.tags.environment=${spring.profiles.active}
4 management.metrics.tags.region=us-east-1
```

 properties

或者通过代码：

```
1 @Configuration
2 public class MetricsConfig {
3
4     @Bean
5     public MeterRegistryCustomizer<MeterRegistry> metricsCommonTags() {
6         return registry -> registry.config().commonTags(
7             "application", "my-app",
8             "environment", "production"
9         );
10    }
11 }
```

 Java

6 集成 Prometheus 与 Grafana 可视化

6.1 Prometheus 简介

Prometheus 是一个开源的监控系统和时间序列数据库。

特点：

- 多维数据模型（基于 Tags）
- 强大的查询语言（PromQL）
- 不依赖分布式存储
- 支持服务发现
- Pull 模式采集数据

6.2 Spring Boot 集成 Prometheus

添加依赖

```
1 <dependency>
2   <groupId>io.micrometer</groupId>
3   <artifactId>micrometer-registry-prometheus</artifactId>
4 </dependency>
```

 xml

配置端点

```
1 # 暴露prometheus端点
2 management.endpoints.web.exposure.include=prometheus,health,info,metrics
3
4 # 可选: 自定义指标前缀
5 management.metrics.export.prometheus.descriptions=true
```

访问指标

```
1 # 查看Prometheus格式的指标
2 curl http://localhost:8080/actuator/prometheus
```

输出示例:

```
1 # HELP jvm_memory_used_bytes The amount of used memory
2 # TYPE jvm_memory_used_bytes gauge
3 jvm_memory_used_bytes{area="heap",id="PS Eden Space",} 2.5769808E7
4 jvm_memory_used_bytes{area="heap",id="PS Survivor Space",} 1234567.0
5 jvm_memory_used_bytes{area="nonheap",id="Metaspace",} 5.4321E7
6
7 # HELP http_server_requests_seconds Time taken to process HTTP requests
8 # TYPE http_server_requests_seconds summary
9 http_server_requests_seconds_count{method="GET",status="200",uri="/api/users",} 1523.0
10 http_server_requests_seconds_sum{method="GET",status="200",uri="/api/users",} 45.678
```

6.3 Prometheus 部署

docker-compose.yml

```
1 version: '3'
2 services:
3   prometheus:
4     image: prom/prometheus:latest
5     ports:
6       - "9090:9090"
7     volumes:
8       - ./prometheus.yml:/etc/prometheus/prometheus.yml
9       - prometheus_data:/prometheus
10    command:
11      - '--config.file=/etc/prometheus/prometheus.yml'
12      - '--storage.tsdb.path=/prometheus'
13      - '--web.console.libraries=/usr/share/prometheus/console_libraries'
14      - '--web.console.templates=/usr/share/prometheus/consoles'
15
16 volumes:
17   prometheus_data:
```

prometheus.yml 配置

```
1 global:
2   scrape_interval: 15s # 每15秒采集一次
3   evaluation_interval: 15s
```

```

4
5  scrape_configs:
6    - job_name: 'spring-boot-app'
7      metrics_path: '/actuator/prometheus'
8      static_configs:
9        - targets: ['host.docker.internal:8080']
10        labels:
11          application: 'my-spring-boot-app'
12          environment: 'development'

```

启动:

```
1 docker-compose up -d
```

 Bash

访问 <http://localhost:9090> 查看 Prometheus UI。

6.4 PromQL 查询语言

基本查询

```

1 # JVM内存使用
2 jvm_memory_used_bytes{area="heap"}
3
4 # HTTP请求总数
5 sum(http_server_requests_seconds_count)
6
7 # 特定接口的平均响应时间
8 rate(http_server_requests_seconds_sum{uri="/api/users"}[5m])
9 /
10 rate(http_server_requests_seconds_count{uri="/api/users"}[5m])

```

promql

常用函数

函数	作用	示例
<code>`rate()`</code>	计算增长率	<code>`rate(http_requests_total[5m])`</code>
<code>`sum()`</code>	求和	<code>`sum(jvm_memory_used_bytes)`</code>
<code>`avg()`</code>	平均值	<code>`avg(response_time_seconds)`</code>
<code>`histogram_quantile()`</code>	分位数	<code>`histogram_quantile(0.95, ...)`</code>
<code>`increase()`</code>	增长量	<code>`increase(errors_total[1h])`</code>

P95/P99 响应时间

```

1 # P95响应时间
2 histogram_quantile(0.95,
3   rate(http_server_requests_seconds_bucket[5m])
4 )
5
6 # P99响应时间
7 histogram_quantile(0.99,

```

promql


```
8 rate(http_server_requests_seconds_bucket[5m])
9 )
```

6.5 Grafana 可视化

I Grafana 简介

Grafana 是一个开源的分析和监控平台，支持多种数据源。

优势：

- 丰富的图表类型
- 灵活的 Dashboard
- 支持告警
- 社区模板丰富

I Docker 部署

```
1 # 在docker-compose.yml中添加
2 services:
3   grafana:
4     image: grafana/grafana:latest
5     ports:
6       - "3000:3000"
7     environment:
8       - GF_SECURITY_ADMIN_PASSWORD=admin
9     volumes:
10      - grafana_data:/var/lib/grafana
11
12 volumes:
13   grafana_data:
```



访问 `http://localhost:3000`，默认账号：`admin/admin`。

I 配置 Prometheus 数据源

1. 登录 Grafana
2. Configuration → Data Sources → Add data source
3. 选择 Prometheus
4. URL: `http://prometheus:9090`
5. Save & Test

I 导入 Dashboard

方式 1：使用社区模板

1. Browse → Import Dashboard
2. 输入模板 ID（如：JVM Micrometer = 4701）
3. 选择 Prometheus 数据源
4. Import

常用模板：

模板名称	ID	用途
JVM Micrometer	4701	JVM 监控
Spring Boot Statistics	12900	Spring Boot 应用
HTTP Server Requests	13619	HTTP 请求监控
Micrometer Spring Boot	14441	综合监控

方式 2：自定义 Dashboard

创建自己的面板，添加 PromQL 查询。

TIP

推荐从社区模板开始，然后根据需求定制。

7 链路追踪基础

7.1 为什么需要链路追踪

在微服务架构中，一个请求可能经过多个服务：

```
1 用户请求 → API Gateway → Order Service → Payment Service → Notification Service
2                                     ↓
3                               Inventory Service
```

问题：

- 哪个服务慢了？
- 请求经过了哪些服务？
- 错误发生在哪一层？

解决方案：分布式链路追踪

7.2 OpenTelemetry

OpenTelemetry 是 CNCF 的可观测性框架，提供统一的 API 和 SDK。

组成：

- Tracing：分布式追踪
- Metrics：指标收集
- Logs：日志关联

优势：

- 厂商中立
- 多语言支持
- Spring Boot 3.x 原生支持

7.3 Spring Boot 集成 Micrometer Tracing

I 添加依赖 (Spring Boot 3.x)

```
1 <!-- Micrometer Tracing -->
2 <dependency>
3   <groupId>io.micrometer</groupId>
4   <artifactId>micrometer-tracing-bridge-brave</artifactId>
5 </dependency>
6
7 <!-- Zipkin导出器 -->
8 <dependency>
9   <groupId>io.zipkin.reporter2</groupId>
10  <artifactId>zipkin-reporter-brave</artifactId>
11 </dependency>
```

NOTE

Spring Boot 2.x 使用 Spring Cloud Sleuth, 3.x 已迁移到 Micrometer Tracing。

I 配置

```
1 # 应用名称 (显示在追踪系统中)
2 spring.application.name=order-service
3
4 # Zipkin服务器地址
5 management.zipkin.tracing.endpoint=http://localhost:9411/api/v2/spans
6
7 # 采样率 (1.0 = 100%采样)
8 management.tracing.sampling.probability=1.0
```

I 自动追踪

Spring Boot 会自动追踪:

- HTTP 请求 (RestTemplate、WebClient)
- 数据库查询
- 消息队列
- 异步任务

无需额外代码!

7.4 手动创建 Span

对于自定义逻辑, 可以手动创建 Span:

```
1 import io.micrometer.tracing.Tracer;
2 import org.springframework.stereotype.Service;
3
4 @Service
5 public class OrderService {
6
7     private final Tracer tracer;
8
9     public OrderService(Tracer tracer) {
10         this.tracer = tracer;
```

```
11     }
12
13     public void processOrder(Order order) {
14         // 创建子Span
15         Span span = tracer.nextSpan().name("validate-order").start();
16
17         try (Tracer.SpanInScope ws = tracer.withSpan(span)) {
18             // 业务逻辑
19             validateOrder(order);
20
21             // 添加标签
22             span.tag("order.id", order.getId());
23             span.tag("order.amount", String.valueOf(order.getAmount()));
24         } finally {
25             span.end();
26         }
27     }
28 }
```

7.5 Trace ID 传播

Trace ID 需要在服务间传递：

HTTP 请求：

```
1 Request Headers:
2   X-B3-TraceId: abc123def456
3   X-B3-SpanId: 789xyz
4   X-B3-ParentSpanId: 456abc
```

消息队列：

```
1 // Kafka消息头自动包含追踪信息
2 kafkaTemplate.send("orders", order);
```

 Java

8 分布式追踪实践

8.1 Zipkin 部署

I Docker 部署

```
1 version: '3'
2 services:
3   zipkin:
4     image: openzipkin/zipkin:latest
5     ports:
6       - "9411:9411"
7     environment:
8       - STORAGE_TYPE=mem # 生产环境使用elasticsearch
```

 YAML

```
1 docker-compose up -d
```

 Bash

访问 <http://localhost:9411> 查看 Zipkin UI。

查看追踪数据

1. 点击“Run Query”
2. 选择 Service Name
3. 查看 Trace 列表
4. 点击 Trace 查看详细调用链

界面展示：

1	Trace: abc123def456
2	
3	API Gateway [] 120ms
4	└─ Order Service [] 80ms
5	└─ Validate [] 20ms
6	└─ Payment [] 40ms
7	└─ Inventory [] 10ms
8	

8.2 Jaeger 部署

Jaeger 是 Uber 开源的分布式追踪系统。

Docker 部署

1	version: '3'	YAML
2	services:	
3	jaeger:	
4	image: jaegertracing/all-in-one:latest	
5	ports:	
6	- "16686:16686" # UI	
7	- "14268:14268" # Collector	
8	environment:	
9	- COLLECTOR_ZIPKIN_HOST_PORT=:9411	

访问 <http://localhost:16686> 查看 Jaeger UI。

Spring Boot 配置

1	# Jaeger exporter	properties
2	management.otlp.tracing.endpoint=http://localhost:4318/v1/traces	

Zipkin vs Jaeger

特性	Zipkin	Jaeger
开发公司	Twitter	Uber
UI 简洁度	简单	功能丰富
存储后端	Memory/MySQL/ES	Memory/Cassandra/ES
社区活跃度	中等	高

适用场景	小型项目	大型微服务
------	------	-------

TIP

小型项目推荐 Zipkin（简单易用），大型微服务推荐 Jaeger（功能强大）。

8.3 追踪数据分析

性能瓶颈定位

通过 Trace 视图可以快速发现：

- 1. 慢服务：哪个 Span 耗时最长
- 2. 并行优化：哪些 Span 可以并行执行
- 3. 调用次数：是否有不必要的重复调用

示例分析

1	Trace: order-creation		
2			
3	OrderController	[████████] 250ms	← 总耗时
4	├ Validation	[██] 20ms	✓ 快速
5	├ DB Insert	[██] 50ms	✓ 正常
6	├ Payment	[██████] 150ms	✗ 慢!
7	├ └ API Call	[██████] 140ms	← 外部API慢
8	└ Notification	[██] 30ms	✓ 正常
9			
10			
11	结论：Payment服务调用的外部API是瓶颈		

错误追踪

Span 会记录异常信息：

```
1 try {
2     callExternalService();
3 } catch (Exception e) {
4     span.tag("error", "true");
5     span.event("exception");
6     throw e;
7 }
```

Java

在 Zipkin/Jaeger 中可以：

- 过滤出错误的 Trace
- 查看异常堆栈
- 分析错误率趋势

9 日志聚合与分析

9.1 为什么需要日志聚合

在分布式系统中，日志分散在多个服务器： ...

问题：

- 如何跨服务追踪一个请求？
- 如何快速搜索特定日志？
- 如何分析日志趋势？

解决方案：集中式日志聚合

9.2 ELK Stack

ELK 是 Elasticsearch、Logstash、Kibana 的缩写。

I 架构

1 应用日志 → Filebeat → Logstash → Elasticsearch → Kibana

组件	作用
Filebeat	日志收集器，轻量级
Logstash	日志处理和转换
Elasticsearch	日志存储和搜索
Kibana	日志可视化

I Docker 部署

```
1 version: '3'
2 services:
3   elasticsearch:
4     image: docker.elastic.co/elasticsearch/elasticsearch:8.11.0
5     environment:
6       - discovery.type=single-node
7       - xpack.security.enabled=false
8     ports:
9       - "9200:9200"
10
11   kibana:
12     image: docker.elastic.co/kibana/kibana:8.11.0
13     ports:
14       - "5601:5601"
15     depends_on:
16       - elasticsearch
17
18   logstash:
19     image: docker.elastic.co/logstash/logstash:8.11.0
20     volumes:
21       - ./logstash.conf:/usr/share/logstash/pipeline/logstash.conf
22     ports:
23       - "5044:5044"
```

YAML

Spring Boot 配置

使用 JSON 格式输出日志（见第 4 章），Filebeat 自动采集并发送到 Logstash。

访问 `http://localhost:5601` 查看 Kibana。

9.3 Loki + Grafana

Loki 是 Grafana Labs 开发的轻量级日志聚合系统。

优势：

- 不索引日志内容，只索引标签
- 存储成本低
- 与 Grafana 无缝集成
- 适合云原生环境

架构

1 应用日志 → Promtail → Loki → Grafana

组件	作用
Promtail	日志收集器
Loki	日志存储
Grafana	日志查询和可视化

Docker 部署

```
1 version: '3'
2 services:
3   loki:
4     image: grafana/loki:latest
5     ports:
6       - "3100:3100"
7     command: -config.file=/etc/loki/local-config.yaml
8
9   promtail:
10    image: grafana/promtail:latest
11    volumes:
12      - /var/log:/var/log
13      - ./promtail-config.yaml:/etc/promtail/config.yaml
14    command: -config.file=/etc/promtail/config.yaml
```

Grafana 中查询日志

使用 LogQL 查询语言：

```
1 # 查询特定应用的日志
2 {app="order-service"}
3
4 # 包含特定关键词
```



```
5 {app="order-service"} |= "error"
6
7 # 排除特定级别
8 {app="order-service"} !~ "level=DEBUG"
9
10 # 统计错误数量
11 sum(count_over_time({app="order-service"} |= "error" [5m]))
```

ELK vs Loki

特性	ELK	Loki
全文搜索	✓ 强大	✗ 有限
存储成本	高	低
学习曲线	陡峭	平缓
适用场景	复杂分析	快速排查
资源占用	高	低

TIP

简单日志排查推荐 Loki（轻量），复杂分析推荐 ELK（功能强大）。

10 告警与通知

10.1 Prometheus Alertmanager

告警规则

创建 `alert.rules.yml`:

```
1 groups:
2   - name: application-alerts
3     rules:
4       # CPU使用率过高
5       - alert: HighCPUUsage
6         expr: process_cpu_usage > 0.8
7         for: 5m
8         labels:
9           severity: warning
10        annotations:
11          summary: "High CPU usage detected"
12          description: "CPU usage is {{ $value }}% for more than 5 minutes"
13
14       # 错误率过高
15       - alert: HighErrorRate
16         expr: rate(http_server_requests_seconds_count{status=~"5.."}[5m]) > 0.1
17         for: 2m
18         labels:
19           severity: critical
20        annotations:
```

```
21     summary: "High error rate"
22     description: "Error rate is {{ $value }} errors/sec"
23
24     # 响应时间过长
25     - alert: SlowResponse
26       expr: histogram_quantile(0.95, rate(http_server_requests_seconds_bucket[5m])) > 2
27       for: 5m
28       labels:
29         severity: warning
30       annotations:
31         summary: "Slow response time"
32         description: "P95 response time is {{ $value }}s"
```

Alertmanager 配置

```
1 route:
2   receiver: 'default-receiver'
3   group_by: ['alertname']
4   group_wait: 30s
5   group_interval: 5m
6   repeat_interval: 4h
7
8 receivers:
9   - name: 'default-receiver'
10     email_configs:
11       - to: 'admin@example.com'
12         from: 'alertmanager@example.com'
13         smarthost: 'smtp.example.com:587'
14         auth_username: 'alertmanager@example.com'
15         auth_password: 'password'
```

10.2 Grafana 告警

配置告警

1. Dashboard 面板 → Alert → Create Alert
2. 设置条件（如：CPU > 80%）
3. 配置通知渠道
4. 保存

通知渠道

Grafana 支持多种通知方式：

渠道	配置难度	适用场景
Email	简单	正式通知
钉钉	中等	国内团队
企业微信	中等	国内团队
Slack	简单	国际团队

Webhook	灵活	自定义
---------	----	-----

10.3 钉钉告警配置

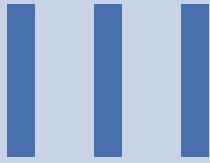
Webhook 方式

```
1 import org.springframework.stereotype.Service;
2 import org.springframework.web.client.RestTemplate;
3
4 @Service
5 public class DingTalkAlertService {
6
7     private static final String WEBHOOK_URL = "https://oapi.dingtalk.com/robot/send?access_
        token=YOUR_TOKEN";
8
9     public void sendAlert(String title, String message) {
10         RestTemplate restTemplate = new RestTemplate();
11
12         Map<String, Object> requestBody = new HashMap<>();
13         requestBody.put("msgtype", "markdown");
14
15         Map<String, String> markdown = new HashMap<>();
16         markdown.put("title", title);
17         markdown.put("text", "## " + title + "\n\n" + message);
18
19         requestBody.put("markdown", markdown);
20
21         restTemplate.postForObject(WEBHOOK_URL, requestBody, String.class);
22     }
23 }
```

集成 Prometheus

通过 Alertmanager 的 Webhook receiver 转发到钉钉。

本章详细介绍了 Spring Boot 的监控与可观测性体系，从 Actuator 端到 Micrometer 指标，从 Prometheus+Grafana 可视化到分布式链路追踪，再到日志聚合和告警通知。结合第 4 章的日志系统，您已经掌握了构建完整可观测性平台的所有关键技术。



微服务与中间件

Chapter 6

微服务架构基础

1 微服务架构概述

2 服务注册与发现

3 配置中心

4 API 网关

5 负载均衡与熔断降级

IV

Spring Boot 源码